

Programming 1

Lab manual to accompany COMP100.

01101110
01100001
01110010
01100101
01101110
01100100
01110010
01100001



01110000
01100101
01110010
01110011
01101000
01100001
01100100

Narendra K. Pershad

Software Group, SIT Dept.

September 2025

ASENT³ >

Centennial College

TABLE OF CONTENTS

0 – Introduction	13
0.a – What is Programming?.....	13
0.a.1 – Programming Paradigms	13
0.b – Python	13
0.b.1 – What can be done with Python?	13
0.b.2 – What Cannot Be Done (or Is Not Ideal) with Python?.....	14
0.b.3 – Why use Python?.....	14
0.b.4 – Why use another language?	15
1 – Getting Started.....	16
Learning outcomes.....	16
1.a – Writing and Running Code	16
1.a.1 – Python REPL.....	17
1.a.2 – Running Python code from the command prompt	18
1.a.3 – Running Python code interactively from the command prompt.....	18
1.a.4 – Running Python as an Executable (Unix/Linux/macOS)	19
1.a.5 – Notebooks	19
1.b – Sending output to the screen: using the print() function	20
1.c – Getting input from the keyboard: using the input() function	20
1.d – Comments.....	21
1.e – Variables	21
1.e.1 – Rules for Naming Variable (enforced by Python)	22
1.e.2 – Naming Conventions (required for this course)	22
1.f – Some useful built-in functions	23
1.f.1 – id()	23
1.f.2 – type()	24
1.f.3 – dir()	25
1.f.4 – help()	25
1.f.5 – eval()	26
1.g – Summary	27
1.h – Non coding exercises	27
2 – Datatypes.....	29
Learning Outcomes	29
Introduction	29
Composite Types	30
Recognizing Literals	30

2.a – None	31
2.b – Numerics.....	32
2.b.1 – Integers.....	32
2.b.2 – Floating Point Numbers.....	34
2.b.3 – Complex Numbers	35
2.b.4 – Fraction (informational)	36
2.b.5 – Decimal (informational)	36
2.c – Sequences:	36
2.c.1 – Strings	36
2.c.2 – Tuple	37
2.c.3 – Range	37
2.c.4 – List.....	37
2.d – Mappings:	38
2.d.1 – Dictionary	38
2.e – Set types	38
2.e.1 – Set.....	39
2.f – Binary types	39
2.g – Enum.....	39
2.h – Checking Data Types.....	39
2.i – Converting Data Types	40
2.i.1 – Implicit	40
2.i.2 – Explicit	41
2.j – Operators.....	42
2.j.1 – Assignment Operators.....	42
2.j.1 – Arithmetic Operators.....	43
2.j.2 – Relational (Comparison) Operators.....	44
2.j.3 – Logical Operators.....	45
2.j.4 – Identity Operators	46
2.j.5 – Membership Operators	46
2.j.6 – Bitwise Operators	47
2.j.7 – Operator precedence	48
2.k – Summary	49
2.l – Non coding exercises	49
3 – Control Structures: Sequential and Conditionals	51
Learning Outcomes	51
Introduction	51
3.a – Sequential	52

3.b – Conditionals	53
3.b.1 – if.....	54
3.b.2 – else	54
3.b.3 – elif.....	54
3.b.4 – and.....	55
3.b.5 – or	55
3.b.6 – not	55
3.b.7 – Ternary statements	55
3.c – Comparison Operators	56
3.c.1 – Greater than (>) operator	56
3.c.2 – Less than (<) operator	58
3.c.3 – Greater than or equal (>=) operator	58
3.c.4 – Less than or equal (<=) operator	59
3.c.5 – Equality (==) operator	59
3.c.6 – Not equals (!=) operator	60
3.d – Logic Operators	61
3.d.1 – Logical ‘and’ operator	61
3.d.2 – Logical ‘or’ operator	62
3.d.3 – Logical ‘not’ operator	62
3.d.3.1 – Bit fiddling (This sub-section covers an advanced topic. You can safely ignore it.)	62
3.d.4 – Best Practice	65
3.e – Match statement	65
3.f – Summary	67
3.g – Non coding exercises	67
4 – Repetition.....	69
4.a – While statements	69
4.b – For loop.....	70
4.b.1 – Iterating Over an Iterable	70
4.b.2 – Using the range object	71
4.b.3 – Using the _ (Discard Variable)	71
4.b.4 – Using the Enumerate Function	72
4.b.5 – Nested Loop	74
4.c – Loop control Keywords.....	74
4.c.1 – break	74
4.c.2 – continue	75
4.c.3 – pass	75
4.d – Infinite Loops	75
4.e – Summary.....	76
4.f – Non coding exercises	77

5 – Functions.....	78
5.a – User-Defined Functions.....	78
5.a.1 – Creating Functions	78
5.a.2 – Calling Functions.....	79
5.a.3 – Functions with Inputs	79
5.a.4 – Functions that explicitly return a value	79
5.a.5 – Variable Scope	80
5.a.6 – Keyword Arguments	80
5.a.7 – Default Arguments	81
5.a.8 – Annotation.....	81
5.a.9 – Recursive Functions	82
5.a.10 – Inner Functions	83
5.a.11 – Higher-order Functions.....	84
5.a.12 – Closures	86
5.b – Documentation	87
5.b.1 – Code Explanation.....	87
5.b.2 – Disabling Code	87
5.b.3 – To-Do Notes.....	88
5.b.4 – Code Maintenance	88
5.b.5 – Code Review	88
5.b.6 – Debugging Information	88
5.b.7 – Best Practices for Writing Comments	89
5.c – Built-in Functions	89
Randomisation	92
range	92
round.....	92
abs	93
5.d – Summary.....	93
5.e – Non-Coding Exercises.....	94
6 – Sequences	96
Immutables	96
Mutables	97
6.a – String	97
6.a.1 – String Creation Methods	98
6.a.1 – String Concatenation	98
6.a.2 – Escaping a String	99
6.a.3 – Raw strings	99
6.a.4 – Combining F-Strings and raw strings	100
6.b – List.....	100
6.b.1 – Creating Lists	100

6.b.2 – Adding Lists Together	100
6.b.3 – Adding an Item to a List.....	101
The in operator.....	101
List Indexing.....	101
List Slicing	101
6.c – Common Sequence Operations	102
6.b.1 – min(), max() sum()	102
6.b.1 – Length.....	102
6.b.1 – Membership test	102
6.b.1 – Concatenation	103
6.b.1 – Repetition.....	104
6.b.1 – Iteration.....	104
6.b.1 – Indexing	104
6.b.1 – Slicing.....	104
6.d – Summary.....	104
6.e – Non coding exercises	106
8 – Sets & Mappings.....	108
8.a – Learning Outcomes	108
8.b – Introduction	108
8.c – Set.....	109
8.c.1 – Creating sets	109
8.c.2 – Common Set Methods	110
Set Operators	115
8.d – Dictionary	115
8.e – Summary.....	116
8.f – Non coding exercises	117
9 – File I/O & Errors.....	118
9.a – Learning Outcomes	118
9.b – Introduction	118
9.c – Working with files.....	119
9.c.1 – Opening a file	119
9.c.2 – Writing to a file	119
9.c.3 – Reading from a file.....	119
9.c.4 – A more elegant way of reading/writing a file	120
9.c.5 – File Modes.....	120
9.d – Operation System File Management	121
9.d.1 – Checking for the existence of a file	121
9.d.2 – Deleting a file.....	121

9.d.3 – Deleting a folder	121
9.d.4 – Creating a folder	122
9.e – Errors	122
9.e.1 – Syntax Error	122
9.e.1 – Name Error	122
9.e.2 – Type Error	123
9.e.3 – Value Error	123
9.e.4 – Index Error	123
9.e.5 – Key Error	123
9.e.6 – FileNotFoundError	124
9.e.7 – Zero Division Error	124
9.e.8 – Error handling	124
9.e.9 – A Practical Example	125
9.f – Summary	125
9.f.1 – File I/O:	125
9.f.2 – Errors:	126
9.f.3 – Error Handling:	126
9.g – Non-Coding Exercises	126
10 – Packages/Modules	128
10.a – Learning Outcomes	128
10.b – Introduction	128
10.c – Modules	128
10.c.1 – Examples of built-in modules	129
10.c.2 – External modules	129
10.c.3 – Creating your own Module	129
10.c.4 – Importing/Loading a Module	130
10.c.5 – Using the Module	132
10.c.6 – Removing a Module	132
10.d – Packages	132
10.d.1 – Package Structure	133
10.d.2 – Creating your own Package	133
10.e – Summary	133
10.f – Non coding exercises	134
11 – Object Oriented Programming	136
11.a – Learning Outcomes	136
11.b – Introduction	136
11.c – Classes and Objects	137
11.c.1 – Defining a Python Class	137

11.c.2 – Instantiating the Class.....	137
11.c.3 – The <code>__init__</code> method	138
11.c.4 – Instance Variables.....	138
11.c.5 – Instance Methods	139
11.c.6 – Class Variables	139
11.c.7 – Class Methods.....	140
11.c.8 – Static Methods.....	141
11.c.9 – Key Concepts	141
11.c.10 – Comparison: Instance vs. Class vs. Static Methods	142
11.d – The Four Pillars of Object-Oriented Programming	142
11.b.1 – Encapsulation	143
11.b.2 – Inheritance	144
11.b.3 – Polymorphism	145
11.b.4 – Abstraction	146
Summary of the Four Pillars	147
11.e – Summary	148
11.e.1 – Quick Reference: Defining a Class	148
11.f – Non coding exercises	149
Appendices	151
a – Programming Paradigms.....	151
a.1 – Imperative Programming.....	151
a.2 – Procedural Programming.....	151
a.3 – Object-Oriented Programming (OOP)	152
a.4 – Functional Programming	152
a.5 – Declarative Programming	152
Why are Programming Paradigms Important?	153
a.6 – The Best Paradigm	153
b – What can be done with Python?	153
b.1 – Machine Learning and Artificial Intelligence	153
c – Why use Python?	154
d – Why you might consider another language?	154
Docstring	154
3.a – None	155
Debugging	155
Types of Errors.....	156
Syntax Errors:	156
Runtime Errors:	156
Logical Errors:.....	157
Using print statement	157

Using logging	158
Using Python Debugger (pdb).....	158
Using IDE Debugger (VS Code)	158
Setting breakpoints	158
Stepping through code	158
Examining Variables	158
Iterators.....	158
a – The Iterator Protocol	158
b – Using Built-in Iterator	158
c – Practical Benefits	159
d – Iterables vs Iterators.....	159
e – Creating Custom Iterators	160
f – When not to Prefer Iterators	160
Generators.....	161
Generator Expressions	161
Using PIP	162
Default Installation	162
Install a specific version	162
Install Extra	162
Install from Source.....	162
Install from Version Control	162
Install from other Package Indexes.....	162
Upgrading packages.....	162
Using a requirements file	162
Freezing dependencies.....	162
CoPilot	162
a – Installing Copilot	162
b – Enabling Copilot	163
c – Disabling Copilot.....	163
d – Tips of using Copilot	163
i – Write clear comments	163
ii – Use descriptiveWrite clear comments.....	163

Virtual Environments	164
a – Creating Virtual Environments	164
b – Activating the Virtual Environment	165
b.1 – On Windows	165
b.2 – On macOS/Linux	166
c – Installing Packages	166
c.1 – Batch Installation	167
d – Deactivating the Virtual Environment	168
e – Deleting a Virtual Environment	168
Regular Expression	169
match()	169
search()	169
findall()	170
finditer()	170
sub()	170
Exception Handling	171
Testing	171
Types of Testing	171
Unit Testing	171
Integration Testing	171
Functional Testing	171
Regression Testing	171
End-to-End Testing	171
Tools	172
unittest	172
pytest	172
doctest	172
Basic Testing Workflow	172
Modules and Packages	172
a – Module	172
a.1 – Importing the module	173
a.2 – Import using alias	174
a.3 – Import only selected objects	174
a.4 – Import everything	174
b – Package	175

c – Library.....	175
d – Framework.....	175
Extending Python.....	175
Install the Python.Net package.....	175
Write the C# code	176
Compile to a DLL.....	176
Load the library and invoke the Greet() method	176
Other languages.....	177
When not to extend python	177
Keywords.....	177
0 – Glossary	179
Assignment.....	179
Indexable	179
Iterable	179
Sequences	179
Collections	179
Mappings.....	179
Method.....	180
Function.....	180
Overloading	180
Inheritance	180
Type Annotation.....	180
Docstring	180
Comments	180
Repl	180
IDE	180
Debugging	180
Operator.....	180
Keyword	180
Module	180
Import.....	180
Algorithm.....	180
Boolean (bool).....	181
Collection.....	181
Comment.....	181
# This is a comment.....	181
Data Type	181
Debugging	181
Dictionary (dict).....	181
Docstring	182

Function.....	182
Method.....	182
Indentation.....	182
Inheritance	182
Iterable	183
Keyword	183
List	183
Loop.....	183
Module	183
Import.....	183
Mutable	184
Operator	184
Sequence	184
Tuple.....	184
Type Annotation.....	185
Variable	185
2.j.7 – Operator precedence	186

0 – INTRODUCTION

0.a – What is Programming?

Programming is the art and science of giving a computer a precise, logical set of instructions to solve a problem or perform a task. It involves analyzing a problem, breaking it down into smaller more manageable steps, and expressing those steps in a way that a computer can understand and execute.

Programming relies on fundamental concepts such as **variables**, **data types** and **controls structures** to build functional software systems such as websites, mobile applications, games, operating systems embedded systems and AI-driven apps.

0.a.1 – Programming Paradigms

A programming paradigm is a set of principles or a style that guides how programs are structured and how problems are solved. It influences which tools and techniques are preferred and how they are combined to produce reliable and maintainable software.

Common paradigms include:

- Imperative
- Declarative
- Functional
- Object-Oriented

These paradigms are discussed in more detail in the appendix of this document.

0.b – Python

Python is a high-level, interpreted, and general-purpose programming language created by Guido van Rossum and first released in 1991. It emphasizes code readability and simplicity allowing programmers to express ideas using fewer lines of code compare to languages such as C++ or Java.

0.b.1 – What can be done with Python?

It excels in areas where developer productivity, readability, and the ability to leverage powerful libraries are more important than raw execution speed or low-level control. It is able to work seamlessly with external libraries.

- **Data Science, Analytics and Visualization:** This is one of Python's biggest strongholds. Libraries such as Pandas, NumPy, Matplotlib and Seaborn makes it a powerhouse for working with data.
- **Machine Learning and Artificial Intelligence:** Libraries like scikit-learn, TensorFlow, PyTorch and Keras are high-performant and can take advantage of specialize hardware designed for AI.
- **Back-End Web Development:** there are excellent libraries such as Django, Flask and FastAPI.
- **Automation, Scription and DevOps**
- **Scientific and Numeric Computing:** Access to high performance libraries like NumPy, SciPy.
- **Web Scraping**
- **Software Prototyping:** It can quickly validate an idea or building a Minimum Viable Product (MVP)
- **Education:** It is the most popular introductory teaching language in universities, colleges and coding bootcamps worldwide

0.b.2 – What Cannot Be Done (or Is Not Ideal) with Python?

Although Python is a very versatile, it is not always the best tools for every tasks. Its limitation typically falls into the areas of performance, system-level programming and constrained environment.

Examples includes:

- **System-level Programming** such as operating systems or device drivers.
- **Real-time or Ultra-High-performance Applications.**
- **Memory-Intensive Tasks** in constrained environments.
- **Browser/Client-Side Web Development.**
- **Statically-Typed and Secure-By-Design Systems.**

0.b.3 – Why use Python?

Python is a powerful, accessible, and extremely popular language that acts as a Swiss Army knife for programmers. It is suitable for beginners and professionals alike, with a massive ecosystem of libraries and a strong global community. Its emphasis on clear and readable code makes it an excellent first language as well as a professional tool used by organizations such as Google, Netflix, Instagram, and NASA.

0.b.4 – Why use another language?

- **CPU Intensive Tasks:** Python being an interpreted and high-level language is slower than compiled languages like C++.
- **Mobile Applications:** Swift, Kotlin, Java Flutter or React Native are better suited.
- **Compile-Time Error Detection:** Python's dynamic typing can allow some errors to surface only at runtime.
- **In-Browser Applications:** JavaScript or TypeScript are required.

1 – GETTING STARTED

This chapter introduces the fundamentals needed to begin working with Python. You will learn how to write and run Python programs using different tools and environments, ranging from interactive sessions to full program execution. Along the way, you'll explore how Python displays output, accepts input from the keyboard, and uses variables to store data.

You'll also be introduced to comments, naming conventions, and a handful of built-in functions that help you understand and explore Python code. By the end of this chapter, you will have the foundation needed to comfortably experiment with Python, read simple programs, and prepare for more advanced topics later in the course.

Learning outcomes

By the end of this chapter, you will be able to:

1. Run Python code in Jupyter Notebooks
2. Display output with `print()` and read keyboard input with `input()`
3. Write comments that add value rather than noise
4. Create Variable that follow Python's rules and this course conventions
5. Use `id()`, `type()`, `dir()`, `help()`, and `eval()` to inspect Python objects

1.a – Writing and Running Code

There are several common ways to run Python code. Each useful in different situations:

- The Python REPL
- Interactive notebooks (VS Code, Jupyter, Google Colab and PyCharm)
- Running a script from the command line
- Executing a script as an executable file

Fun fact

A Pythonista is someone who enjoys using Python – whether they're a beginner, expert, or somewhere between.

Method	Best for	Main limitation
REPL	Quick experiments, testing one idea, debugging	Awkward for anything longer than a few lines

Command Line	Running a finished program	When the session ends, you cannot inspect anything afterwards
Command Line with -i	Debugging a finished program	Still requires a terminal
Executable script	Distributing to end users	Unifamily systems only
Notebook	Teaching, demonstrations, data work	Not a way to ship software

1.a.1 – Python REPL

The Python REPL is the simplest and most flexible ways to run Python code. It lets you write one line at a time and get immediate feedback, which makes it popular with both beginners and veteran *Pythonistas*.

To use the REPL, you start the python interpreter. This opens a command prompt similar to your operating system shells. This is a **Read-Evaluate-Print Loop**.

- Python **reads** your input
- **evaluates** it
- **prints** the result back to the screen
- and **loops** around to do it again

Note

In the REPL, Python prints the value of any expression automatically — you do not need `print()`. In a script, it does not. This surprises almost everyone at least once.

You can use this technique to incrementally build substantial python applications or parts of applications. You are able to examine each object at your leisure. You may also debug problematic code this way.

The downside is that it is not convenient for larger/longer block of code or distributing code to end users or when typing is challenging.

To leave the REPL, type `exit()` or press `Ctrl-Z` then `Enter` on Windows, or `Ctrl-D` on Unix, Linux, and macOS.

Below is a sample session in the python **repl** in a windows 10 environment:

Copyright (C) Microsoft Corporation. All rights reserved.

```
PS C:\progs\new100\labs> python
Python 3.10.7 (tags/v3.10.7:6cc6b13, Sep  5 2022, 14:08:36) [MSC v.1933 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 6
8
>>> 8 / 3
2.6666666666666665
>>>
>>> hi = 'hi '
>>> hi = hi * 4
>>> hi
'hi hi hi hi '
```

1.a.2 – Running Python code from the command prompt

Once your program is complete and behaving correctly (ideally 95-100% according to specifications), you typically run it from the command line.

```
«path_to_python» «name_of_code_file»

>>> python lec33_docstring.py
```

1.a.3 – Running Python code interactively from the command prompt

The above technique, terminates the python session after the python file is processed. This means you can't inspect variables or object afterwards.

If you use the `-i` switch, Python keeps the session alive. This allows you to examine variables, functions and objects created with the program.

```
«path_to_python» «name_of_code_file»

>>> python -i lec33_docstring.py
```

This is especially useful for debugging and exploration.

1.a.4 – Running Python as an Executable (Unix/Linux/macOS)

In this method of running python code there are two steps involved, changing the permission of the file and then inserting the path to the python interpreter in the first line of the code file.

```
chmod 755 «name_of_code_file»  
or  
chmod a+x «name_of_code_file»
```

And then modifying the code file, insert the following at the top of the file:

```
#!/«path_name_of_python_interpreter»
```

This is probably the best and the most professional way of running or distributing your code to everyone from the novice user to the seasoned developer.

This is one of the most professional ways to distribute Python programs to users of all experience levels.

Interesting fact

This approach works only on operating systems that support file permissions (Linux, Unix, macOS). On Windows, you can achieve a similar result using batch files or PowerShell scripts.

1.a.5 – Notebooks

Notebooks are quickly becoming one of the most popular ways to write, run and demonstrate Python code.

A notebook is made up of three types of cells:

- **Markdown cells** for text and explanations
- **Code cells** containing Python code
- **Output cells** that show the results of running code

Notebooks feels like browsing a live textbook: you can read the theory and see the code, run it, and immediately view the results – all in one place. You can also modify the code and re-run to observe changes.

Many online platforms provide pre-configured environments with the required libraries already installed. This is especially helpful for beginners or when working with complex setups like machine learning or image processing.

Popular notebook platform includes:

- Jupyter
- Binder
- Kaggle
- Google
- Microsoft
- CoCalc
- Datalore.

1.b – Sending output to the screen: using the `print()` function

The `print()` function displays output on the console. While it's simple to use, it also supports more advanced options that we will explore later in the semester.

```
>>> print(4)                                #displays the integer literal 4
4

>>> print(3 + 0.141592)                     #displays the float literal
3.141592

>>> print('Hello World')                   #displays the string literal
Hello World

>>> print(name)                             #display the contents of variable name
Narendra Pershad
```

1.c – Getting input from the keyboard: using the `input()` function

The `input()` function displays a prompt and waits for the user to type something. The results is always returned as a string, so it's usually stored in a variable for later use.

```
>>> name = input("What's your name? ")
```

```
What's your name? Narendra      #user response is shown in bold face

>>> print(name)(my_list)
Narendra
```

In the next chapter, we will learn how to convert this text input to numbers and other data types.

1.d – Comments

Comments are lines of text that are ignored by the interpreter. They are used to improve the readability of the code for coders themselves or other developers. For single-line comments, use the “#” symbol, for multi-line comments use either triple single or double quotes.

Fun fact

Comments are not written in the repl.

```
#this is a single-line comment

'''
This is a multi-line comment.
It can span more than
one line
'''
```

Because Python code is generally easy to read, avoid unnecessary comment that restates the obvious. Use comments sparingly to explain why something is done, and not what the code already makes clear

1.e – Variables

At the earliest days of computing, when computer scientist wants to use a piece of memory, they have to use a memory address to refer to that location, this is both tedious and error-prone. Shortly afterwards, language designers allow programmers to assign a name to a memory location. So instead of referring to numeric memory addresses the programmer uses descriptive names to manipulate values.

A variable gives a name to a piece of data (or location of that data). Like a box with a label, you may access what's inside the box if you know the name of the box.

```
name = 'Narendra'
age = 12
```

Once a variable is created in the REPL, it remains until the current session is destroyed. You can remove a variable using the `del` keyword.

1.e.1 – Rules for Naming Variable (enforced by Python)

- Variable must start with a letter or underscore
- Can not contain spaces
- Cannot be a python keyword
- Cannot contain include symbols like '!', '@', '\$', '#', or '%'
- Avoid using name of built-in function 'list', 'str', 'sum', 'int', or 'len'.
- Avoid starting name with double underscores.

Interesting fact

The following will list all of python keywords:

```
help('keywords')
```

```
>>> student_1 = 'Justin Trudeau'
>>> list = [3]                                #legal, but very bad idea
>>> first_name = 'Narendra'                #not legal
```

1.e.2 – Naming Conventions (required for this course)

These are not enforced by Python, They are enforced by your instructor, and by many professional team that you might be a part of.

Fun fact

C# uses camelcase for variables and pascalcase for methods.

- Use descriptive names
- Avoid single letter unless they are meaningful in the context such as the body of a loop.
- Use **snake-casing** for variables
- Use **all caps** for constants.
- Avoid name that shadow built-in function : list, str, sum, int, len, type, id
- Avoid starting variables with single or double underscores.

```
>>> first_name = 'Narendra'

>>> GST = 0.14                                #suggest to the programmer GST is a
constant
>>> __first_name = 'Narendra'                 #legal but not recommended
>>> a = 3                                       #legal but not descriptive
```

1.f – Some useful built-in functions

Python includes many built-in functions that are available without any setup. These functions solve common problems and save time.

FYI

We will cover function formally later in chapter 5.

In addition to `print()` and `input()` here are some frequently used ones.

1.f.1 – `id()`

The `id()` function returns a unique identifier for an object. In CPython, this usually corresponds to the object's memory address.

The following code illustrates that `int` are immutable and `list` are mutable.

```
>>> a = 3
>>> id(a)
1275421851952                                #address of the int a

>>> a = a + 7
>>> id(a)
1275421852176                                #address of the int a (different)
                                           # ask your instructor to explain this

>>> b = []
>>> id(b)
1275429377408                                #address of the list b

>>> b.append(3)                               # adds an item to the list
>>> b
[3]
```

```
>>> id(b)
1275429377408                                #address of the list b (identical)
                                              # ask your instructor to explain this
```

1.f.2 – type()

The `type()` function tells you the data type of an object:

FYI

We will cover datatypes in the next chapter.

```
>>> type(3)
<class 'int'>

>>> type(3.1415)
<class 'float'>

>>> type('Narendra')
<class 'str'>

>>> type(True)
<class 'bool'>

>>> type([])
<class 'list'>

>>> type((4,))
<class 'tuple'>

>>> type({'a'})
<class 'set'>
```

1.f.3 – dir()

The `dir()` function shows all the objects that are currently in scope. The first usage shows the objects in the global scope and the second usage show the objects in the `__builtins__` namespace.

FYI

A more powerful way to explore Python's built-in documentation system is to use the command:

`python -m pydoc -b`

```
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'a', 'b']
```

```
>>> dir(__builtins__)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',
...
'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super',
'tuple', 'type', 'vars', 'zip']
```

This function is very useful for introspection. It allows you to list the attributes and methods of an object especially in the repl.

1.f.4 – help()

The `help()` function launches Python built-in help system. Invoking the `help()` function with `str` and `str.split` arguments.

```
>>> help(str)
Help on class str in module builtins:

class str(object)
|   str(object='') -> str
|   str(bytes_or_buffer[, encoding[, errors]]) -> str
|
|   Create a new string object from the given object. If encoding or
|   errors is specified, then the object must expose a data buffer
```

```
>>> help(str.split)
Help on method_descriptor:

split(self, /, sep=None, maxsplit=-1)
    Return a list of the substrings in the string, using sep as the
    separator string.

    sep
        The separator used to split the string.
```

It provides interactive access to Python's documentation. It is useful for quickly getting information about modules, classes, functions, keywords and other Python objects.

1.f.5 – eval()

The `eval()` function evaluates strings as Python code and returns the result.

Warning

Never call `eval()` on input you did not write yourself.

While it can be useful in controlled situations, **never use `eval()` on untrusted input**. It can execute malicious code and should be avoided in production programs.

```
>>> eval('2 + 3')
5

>>> list_string = "[1, 2, 3, 'hello', 4.5]"
>>> my_list = eval(list_string)

>>> print(my_list)
[1, 2, 3, 'hello', 4.5]

>>> print(type(my_list))
```

1.g – Summary

In this chapter, we explored the basics of getting started with Python and the different ways Python code can be written and executed. We learned that Python programs can be run using the REPL, from the command line, as executable scripts on Unix-based systems, or through interactive notebooks such as Jupyter and Google Colab. Each approach serves a different purpose, from quick experimentation to professional code distribution.

We introduced the Python REPL as an interactive environment that allows immediate feedback, making it ideal for learning, testing, and debugging small pieces of code. We also saw how Python scripts can be executed from the command prompt, including the use of the `-i` switch to keep the session alive for further inspection.

The chapter covered basic input and output using the `print()` and `input()` functions, emphasizing that user input is always received as text. We discussed comments and their role in improving code readability, along with best practices for using them effectively.

We examined variables as named references to data, reviewed the rules and conventions for naming them, and introduced the idea of constants. Finally, we explored several useful built-in functions such as `id()`, `type()`, `dir()`, and `help()`, which help with understanding objects, data types, and Python's built-in features.

By the end of this chapter, you should be comfortable running Python code in multiple environments, displaying output, accepting user input, working with variables, and using basic built-in tools to explore and understand Python programs.

1.h – Non coding exercises

1. What does REPL stand for, and what are the four things it does?
2. Name two advantages and two disadvantages of using the REPL.
3. What does the `-i` switch do when running a Python script from the command line?

4. What two steps are required to run a Python file as an executable on Unix/Linux/macOS?
5. What are the three types of cells in a notebook, and what is each used for?
6. What does `print()` do?
7. What data type does `input()` always return?
8. What is a comment, and what symbol starts a single-line comment?
9. What is a variable?
10. List three rules for naming a variable in Python.
11. What naming convention does this course use for variables and for constants?
12. What does `type()` do?
13. What does `id()` return?
14. What does `dir()` do?
15. What does `help()` do?
16. What does `eval()` do?
17. Why is `list = [3]` considered illegal or bad practice?

2 – DATATYPES

Learning Outcomes

By the end of this chapter, you should be able to:

- Identify Python's built-in data types.
- Distinguish between primitive and composite data types.
- Determine the type of a value using the `type()` function.
- Convert values from one data type to another.
- Use common operators with different data types.
- Understand how Python handles truth values and numeric types.

Introduction

When learning a new programming language, one of the first things to understand is the set of **data-type** that the language supports.

A data type defines the kind of values that can be stored and manipulated by a program. Understanding data types is important because they determine

- What kind of information a variable can store.
- What operations can be performed on that value.
- How Python represents the value internally.

A solid grasp of data types is essential for writing program that are more **robust**, **readable** and **easier to debug**, and it helps you become more fluent in the language.

Unlike many other statically typed languages you may have studied (such as C# or Java), **Python is a dynamically typed**. Other common dynamically-typed languages include Ruby and JavaScript™). This means that a variable is permanently associated with a single type. A variable may refer to different types over the life time of a program.

FYI

A statically typed language is a programming language where variable types are checked at compile-time (before the program runs), rather than at runtime.

In this chapter, we focus on Python's primitive (basic data) types including:

Type	Example
<code>int</code>	42

float	3.14
bool	True
str	'Hello'
None	None

Composite Types

Composite types can contain multiple values:

Type	Example
list	[1, 2, 3]
tuple	(1, 2, 3)
set	{1, 2, 3}
dict	{'name': 'Narendra'}

These composite types are introduced here but explored in more detail later in the course.

Recognizing Literals

In many cases, you can determine the type of a value simple by looking at it:

FYI

Although both a set and a dict are delimited by curly braces, the items in a dict are key-value pairs whilst in a set they are simply individual items.

Literal	Data Type
10	int
3.14	float
True	bool
None	NoneType
[1, 2, 3]	list

(1, 2, 3)	tuple
{1. 2. 3}	set
{'a': 1}	dict

Being able to recognize literals quickly will make it easier to read and understand python code.

2.a – None

None is a special constant in Python that represents the absence of a value. It is **not equal to anything** except to another None. This is similar to the Null object in C#.

```
>>> None == 0
False

>>> None == []
False

>>> None == False
False

>>> x = None
>>> y = None
>>> x == y
True
```

None is its own object with its own type:

```
>>> type(None)
<class 'NoneType'>
```

2.b – Numerics

Computer perform calculations using numeric data types. Numbers object in Python are **immutable**, meaning their values cannot be changed in place. When a new value is assigned, Python creates a new object in memory.

There are three types of numbers available without any imports:

2.b.1 – Integers

Integers are whole numbers with unlimited precision. Python support integer literals in several bases:

Fun fact

C# has 13 primitive numeric types

- Base 10 (decimal) – default
- Base 2 (binary) – prefix `0b`
- Base 8 (octal) – prefix `0o`
- Base 16 (hexadecimal) – prefix `0x`

Some int literals

```
>>> 7                                #base 10 literal
7

>>> 0b100                            #base 2 literal (prefix 0b)
4

>>> 0o27                             #base 8 literal (prefix 0o)
23

>>> 0xf                             #base 16 literal (prefix 0x)
15
```

Python also allows underscores as **digit separators** to improve readability.

```
>>> 0b100_0000                      #binary literal with digit separator
64

>>> 100_000                         #decimal literal with digit separator
```

```
100000
```

Integer can grow arbitrarily large:

```
>>> 1234567890123456789012345678901234567890 + 1
1234567890123456789012345678901234567891
```

Integers are commonly used for:

- Counting
- Index positions
- Loop control variables
- Storing id's
- Performing arithmetic

2.b.1.2 – bool

This is an int sub-type, having only two literal bool values: **True** (1) or **False** (0)

Fun fact

The objects **True**, **False** and **None** are considered immortal in python.

```
.
>>> False + 0
0

>>> True + 0
1

>>> True + True
2
```

Unlike C#, every python object has a truth value. The following are considered False:

- None,
- False,

- Zero numeric type: 0, 01, 0.0, 0j
- Any empty sequence: '', {}, []

All other values are considered True

```
>>> bool([])           #empty list
False
>>> bool([0])
True
>>> bool('')           #empty string
False
>>> bool('0')
True
>>> bool(0)            #integer zero
False
>>> bool(3.1415)
True
```

Boolean values are heavily used in:

- Decision making
- Loops
- Comparisons

2.b.2 – Floating Point Numbers

Floats represent number with decimal places. Internally, they are stored as a **binary fraction**, which means not all decimal values can be represented exactly.

```
>>> .1 + .2
0.30000000000000004
```

Fun fact

Most languages are not able to represent float value exactly. The Decimal type in the decimal module is able to represent a decimal number exactly.

Any division produces a float.

```
>>> type(4 / 2)
<class 'float'>
```

Python has **only one floating-point type**, unlike C#, python only has a single float type.

You should avoid using float for high precision arithmetic as required for financial applications. Use the decimal module instead.

```
10., .001, 1e100, 3.14e-10
3.141_592                #approximate value of pi
5.972e24                 #mass of the earth
9.1093837e-31            #mass of an electron in kg
```

Floats are useful for:

- Scientific calculations
- Measurements
- Percentages
- Temperatures

2.b.3 – Complex Numbers

Complex numbers consist of a **real** and an **imaginary** part. In Python a complex literal must include an imaginary component, indicated by **j**. The following are some examples of complex literals:

```
3.14j, 10.j, 10j, .001j, 1e100j, 3.14e-10j
1.414_213j                #root of -2
```

Fun fact

The Italian mathematician **Gerolamo Cardano** first used complex number in the 16th century.

2.b.4 – Fraction (informational)

This section is for information only. The fraction datatype is available via an explicit import only. You are able to use numbers as ratios of integers.

2.b.5 – Decimal (informational)

This section is also for information only. The decimal datatype is available via an explicit import only.

2.c – Sequences:

Python provides several sequence types, which share some common behaviors such as being **iterable** and **indexable**:

- `str`
- `tuples` -> `()`
- `lists` -> `[]`
- `range` -> `range(5)`

`stack` and `queue` will not be covered in this course

2.c.1 – Strings

Strings are immutable sequences of Unicode characters. It is delimited by matching quotes:

- Single quotes `'Hello'`
- Double quotes `"Hello"`
- Triple quotes. `'''Hello'''`

In this course, we will use a single quote for strings with a few exceptions. Triple quotes (either single or double) are used for multi-line strings such as docstrings.

```
a = 'Hello '  
b = "world"  
c = '''This string  
can span multiple  
lines.'''
```

2.c.1.1 – F-Strings

f-strings allow variables and expressions to be embedded directly into text.

```
>>> days = 365
>>> print(f'There are {days} in a year')
There are 365 in a year
```

2.c.2 – Tuple

Tuples are immutable sequences. Once created, their contents cannot be changed. The items do not have to be the same type.

```
>>> a = ()                                #verbatim definition
>>> a
()

>>> b = (1)                               #this will not do a single item tuple
>>> b
1

>>> b = (1, )                             #now you have a single item tuple
>>> b
(1,)
```

2.c.3 – Range

range represents a sequence of integers commonly used in loops.

```
>>> range(4)
range(0, 5)
```

2.c.4 – List

Lists are mutable sequences and are the most commonly used collection type in Python. It serves the purpose of arrays in other languages and it may contain any type of members. The following statements will return lists:

```
>> a = []                                #verbatim definition
>> a
[]

>> b = 'Hello world'.split()             #list from calling split on a string
>> b
['Hello', 'world']
```

2.d – Mappings:

2.d.1 – Dictionary

A dictionary stores a collection of **key-value pairs**.

- Key must be **unique**, **hashable** and **immutable** such as strings, numbers, or tuples. A list may not be used as a key.
- Values may be any python object
- Dictionaries does **not** support positional indexing, only key-indexing.

This data type is flexible and powerful and is extensively used in the language and will be heavily used in this course.

```
#there are lots of ways of creating a dictionary
>>> a = {'red': 'Danger', 1: 'first', 'second': 2, ('a', 'set'): 'a tuple
is my key'}

>>> b = dict(zip(['ilia', 'hao', 'yin'], ['Ilia Nika', 'Hao Lac', 'Yin
Li']))

>>> c = dict([('arben', 'Arben Tapia'),('patrick','Patrick Gignac')])

>>> d = dict(nar='Narendra Pershad', joanne='Joanne Filotti')
```

2.e – Set types

Sets unordered collection of unique elements. They are iterable but not indexable

Common uses:

- Fast membership testing

- Removing duplicates from a sequence
- Mathematical operations such as intersection, union, difference and symmetric difference.

Set (mutable) and Frozen sets (immutable).

2.e.1 – Set

A sequence of unique immutable python objects. Sets, list or dict cannot be in a set. Set does not support indexing.

```
>>> {1, 2, 3}                                #immutable sequence of bytes
{1, 2, 3}
```

Mimics a Mathematical set such that no duplicates are allowed.

Only immutable object may be elements of a set.

2.f – Binary types

These types are used to handle binary data. They include the following:

```
>>> a = b'ABC'                                #immutable sequence of bytes
>>> print(a)
b'ABC'
>>> b = bytearray([65, 66, 67]) #mutable sequence of bytes
>>> print(b)
bytearray(b'ABC')
```

2.g – Enum

This datatype is available via an explicit import. We will not be using this type in Programming 1.

2.h – Checking Data Types

Use the `type()` function to inspect the data type of a value.

```
>>> type('Hello')
```

```
<class 'str'>

>>> type(3)
<class 'int'>

>>> type(True)
<class 'bool'>

>>> type(3.141592)
<class 'float'>

>>> type([])
<class 'list'>

>>> type(())
<class 'tuple'>

>>> type({1,})
<class 'set'>

>>> type({})
<class 'dict'>
```

2.i – Converting Data Types

Most languages provide a mechanism to convert from one data type to another type either through implicit or explicit casting. You can convert a variable from one data type to another.

2.i.1 – Implicit

This kind of conversion is automatic e.g.:

```
a = 3.15                #this is a float
b = 5                   #this is an int
c = a + b               #b is up cast to a float before addition
```

```
#the following does not work (it does work in C#)
a = 'Hello'                #this is a str
b = 5                      #this is an int
c = a + b                  #this does not work
```

2.i.2 – Explicit

Explicit casting is done by constructor methods such as `int()`, `float()` and `str()`:

2.i.2.1 – Converting to string:

```
str()
n = 354
print(str(new_n))          #result 354
```

2.i.2.2 – Converting to int:

```
>>> int(3.14)              #truncates down the float
3
>>> int(-1)
-1
>>> int(0b110)             #binary literal
6
>>> int('110', base=2)
6
>>> int('fe', base=16)
254
>>> int('11', base=12)
13
```

This converts a number or string to an integer assuming base 10. If you would like to specify an alternate base, then the argument must be a string. The string literal must be a legal string: 'one' is not a legal number and will cause a `ValueError` to be raised.

2.i.2.3 – Converting to float:

```
>>> float(-1)
-1.0
>>> float('3.1415')
3.1415
>>> float(1e8)
100000000.0
>>> float(1.2e-3)
0.0012
```

Other conversions can be done with: `tuple()`, `list()`, `set()`, `dict()`, `ord()`, `chr()`, `hex()`, `oct()`, `bin()`

2.j – Operators

Operators usually takes two operands to create expressions. This expression is evaluated to gives a result. The operator determines the type of operands as well as the resulting value. Unary operators only require a single operand whilst ternary operators require three operands.

2.j.1 – Assignment Operators

This operator binds a value to a memory location (name). i.e it sets the value of a variable. The left-hand side value of the operator is bound to the variable on the right-hand side. If the left side is an expression, then it is evaluated and the resulting value is set to the variable on the right-hand side.

```
>>> a = 5          #a is 5

>>> a = 4 * 3      #a is 12

>>> a *= 3         #a is 3 times a or 36

>>> a = b = c = 0  #a, b, and c is set to 0
```

```
>>> a, b, c = 0, 1, 2      #a is 0, b is 1 and c is 2

>>> a, b = b, a          #values of a and b are swapped

>>> a = [0, 1, 2, 3, 4]
>>> b, *c = a
>>> b
0
>>> c
[1, 2, 3, 4]

>>> *b, c = a
>>> b
[0, 1, 2, 3]
>>> c
4
```

2.j.1 – Arithmetic Operators

You can do mathematical calculations with Python as long as you know the right operators. The order of precedence of the operator is listed in order.

```
>>> 5 ** 2              #exponent
25

>>> 4 * 3               #multiply
12

>>> 18 / 7              #divide
2.5714285714285716

>>> 18 // 7             #integer divide
2
```

```
>>> 18 % 7          #modulus
4

>>> 2 + 7           #add
9

>>> 9 - 4           #subtract
5
```

When working with different numeric types, Python will up-convert the lower precision value to a float and then perform the computation. Often the result of such arithmetic is a float.

2.j.1.1 – The Modulo Operator

Often, you'll want to know what is the remainder after a division. e.g. $4 \div 2 = 2$ with no remainder but $5 \div 2 = 2$ with 1 remainder The modulo does not give you the result of the division, just the remainder. It can be really helpful in certain situations, e.g. figuring out if a number is odd or even.

```
5 % 2                #result is 1
```

2.j.2 – Relational (Comparison) Operators

These operators are normally used to compare two values of the same type. The result is a boolean value. You may also use these operators with string type values.

```
>>> 3 == 4          #equal
False

>>> 3 != 4          #not equal
True

>>> 3 > 4           #greater than
False

>>> 3 < 4           #less than
True
```

```
>>> 5 >= 3          #greater than or equal to
True

>>> 5 <= 3          #less than or equal to
False
```

2.j.3 – Logical Operators

Use to compare two values normally of the same type. The result is a bool. You may also use these operators with string type values.

```
>>> True and True    #True only if both operands are True
True

>>> True and False
False

>>> True or True
True

>>> True or False
True

>>> False or False   #False only if both operands are False
False

>>> not True          #inverts the value of the single operand
False

>>> not False
True
```

2.j.3.1 – Short-circuit evaluation:

If the value of an expression can be computed without evaluated all the part, then the parts are not evaluated.

```
>>> x = 0
>>> y = 5
>>> (y != 0) and (x / y > 0)           # division never happens
```

2.j.4 – Identity Operators

The identity operator (**is**) is used to compare objects. If they have the same memory location, then it returns **True**. If that do not have the same location and even if they are equal, it will return **False**.

```
>>> a = [3]
>>> b = [3]
>>> a == b           #compares the values
True

>>> a is b           #compares the memory addresses
False

>>> id(a)
2354776320832

>>> id(b)
2354776273920
```

2.j.5 – Membership Operators

This can be used to test if a value is present in a sequence (string, list, tuple or dict).

```
>>> 'a' in 'Narendra Pershad'
True

>>> 'q' in 'Narendra Pershad'
False

>>> 'hello' in ['hello', 'world']
True
```

```
>>> 1 in {'one': 1, 'two': 2, 'three': 3}
False

>>> 1 not in {'one': 1, 'two': 2, 'three': 3}
True

>>> 'one' in {'one': 1, 'two': 2, 'three': 3} #for dict, the in operator
only check the keys
True
```

2.j.6 – Bitwise Operators

This category of operators requires two integer operands (except ~ not) and it returns an integer. It operates at the bit-level, so the results are not immediately understandable. However, examining the results (and operands) as binary value will make the operators more palatable.

```
>>> 9 << 2
36

>>> print(f'{bin(0b01001 << 2)}')
0b100100

>>> print(f'{bin(0b01001 << 3)}')
0b1001000

>>> print(f'{bin(0b01001 >> 2)}')
0b10

>>> print(f'{bin(0b01001 & 0b00101)}')
0b1

>>> print(f'{bin(0b01001 | 0b00101)}')
0b1101

>>> print(f'{bin(0b01001 ^ 0b00101)}')
0b1100
```

```
>>> print(f'{bin(~0b01001)}')
-0b1010
```

2.j.6.1 – The augmented Operator

Augmented assignment provides a short way to perform a binary operation (arithmetic, boolean or bitwise) and assigning the results back to one of the operands. You can also use any of the other mathematical operators e.g. -=, +=, *=, /=, //=, **=, |=, >>=, <<=, %=, or ^=

```
my_number = 4
my_number += 2      #my_number is now is 6
```

2.j.7 – Operator precedence

If you have an expression with more than one operator, then python uses a set of internal rules to decide the order that the computation must follow. The table below list the various operators in descending order of precedence.

Operator	Description
()	Parentheses
**	Exponentiation
+x -x ~x	Unary plus, unary minus, and bitwise NOT
* / // %	Multiplication, division, floor division, and modulus
+ -	Addition and subtraction
<< >>	Bitwise left and right shifts
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR

<code>==</code> <code>!=</code> <code>></code> <code>>=</code> <code><</code> <code><=</code> <code>is</code> <code>is not</code> <code>in</code> <code>not in</code>	Comparisons, identity, and membership operators
<code>not</code>	Logical NOT
<code>and</code>	AND
<code>or</code>	OR

2.k – Summary

This chapter introduces Python’s core data types and how they behave. Python is a **dynamically typed** language, meaning variables are not locked to a single data type and can change during a program’s execution.

You learn about **primitive data types**, including `None`, numeric types (`int`, `float`, `bool`, `complex`), and strings. `None` represents the absence of a value, numeric types are immutable, integers have unlimited precision, floats represent decimal values with limited precision, and `bool` is a subtype of `int`. Python evaluates all objects as either *truthy* or *falsy*, which is important for conditionals.

The chapter also introduces **composite data types** such as sequences (`str`, `list`, `tuple`, `range`), mappings (`dict`), and sets. Sequences are iterable and indexable, with Python supporting negative indexing. Lists are mutable, tuples are immutable, dictionaries store key–value pairs, and sets hold unique, unordered elements.

You explore **type checking** using `type()` and **type conversion**, both implicit and explicit, using constructors like `int()`, `float()`, and `str()`.

Finally, the chapter covers **operators**, including assignment, arithmetic, comparison, logical, identity, membership, and bitwise operators, along with operator precedence. Understanding these concepts forms the foundation for writing correct, readable, and efficient Python programs.

2.l – Non coding exercises

1. What is a data type?
2. Why is understanding data types important for programmers?
3. What does it mean that Python is dynamically typed?
4. List the data types covered in this chapter.
5. List the composite type introduced in this chapter.

6. What is the purpose of the None value?
7. Which value represents the absence of a value?
8. Since None means nothing, it is the same as 0. (true or false?)
9. What is a truthy value?
10. List some literals that are considered truthy.
11. List some literals that are considered falsy.
12. Which data type is used to store text?
13. What is the purpose of type conversion?
14. Which function is used to determine a value's type?
15. Why should floats be used carefully in financial applications?
16. The expression '10' + '5' produces 15. (true or false?)
17. What is the result of 5 + 3?
18. What is the result of 10 // 3?
19. What is the result of 10 % 3?
20. What is the result of True + True?
21. What is the result of False + 1?
22. What is the result of 5 > 3?
23. What is the result of 5 == 3?
24. What is the result of not False?

3 – CONTROL STRUCTURES: SEQUENTIAL AND CONDITIONALS

Learning Outcomes

By the end of this chapter, you will be able to:

- **Define** control structures and explain their role in determining the flow of execution in a program.
- **Differentiate** between the four main categories of control structures: sequential, conditional/branching, repetition/looping, and function/method/procedure.
- **Describe** how sequential execution works and explain why it is the foundation of all programs.
- **Construct** `if`, `elif`, and `else` statements to implement two-way and multi-way branching.
- **Apply** comparison operators (`>`, `<`, `>=`, `<=`, `==`, `!=`) correctly to values of different types, including numbers, strings, booleans, and lists.
- **Combine** conditions using the logical operators `and`, `or`, and `not`, and explain short-circuit evaluation.
- **Write** ternary expressions for simple conditional assignments and recognize when they reduce readability.
- **Explain** the purpose of bitwise operators (`&`, `|`, `^`, `~`, `<<`, `>>`) and identify scenarios where they are appropriate.
- **Use** the `match` statement (Python 3.10+) for multi-way branching, including literal patterns, OR patterns, guards, and structural patterns.
- **Identify** and describe keywords that alter control flow (`break`, `continue`, `return`) and their effect within control structures.
- **Recognize** the role of indentation in Python and the risks associated with inconsistent whitespace when pasting code.
- **Evaluate** when to use `match` versus `if-elif` chains based on readability and structural complexity.

Introduction

Control structures are fundamental programming constructs that determine how and in what order your code executes. They allow you to control the flow of your program rather than having it simply run from top to bottom in a straight line.

Understanding control structures allows you to

- **Predict** program behavior

- **Write** flexible programs
- **Build** logic that responds to different inputs and situations

The main control structures:

1. **Sequence/Sequential:**
The statements are processed in order with no skips or jumps.
2. **Conditional/Branching:**
Certain blocks of code are executed **only if** a condition is true
3. **Repetition/Looping/Iteration:**
Statements or blocks of statements that are executed zero or more times
4. **Function/Method/Procedure:**
A named block of code that runs only when called

Control structures may be **nested** i.e. you may put a loop in an `if` statement or put a switch statement inside of a loop to achieve complex behaviors. However, in this course we will not define a function inside any other controls structure including itself.

Keywords that Alter Control Flow

The following keywords alter the normal flow inside of control structures:

- **break** – exits a loop early.
- **continue** – skips to the next iteration of a loop.
- **return** – exits a function and optionally returns a value

Each control structure adds expressive power to the language.

3.a – Sequential

Sequential execution is the simplest and is the default behavior of a program. Statements execute one after another, exactly one, in order they appear.

This guarantees that all the statements will be processed exactly once. It serves as the foundation of most processing techniques.

Because all the statements are processed in the same order this control structure will do the exact thing every time it is executed (i.e. behave the same way every time it is processed). So additional control structure is needed to build any substantial program i.e., one that can do at least two or more tasks or to behave differently every time you run it.

```
#set of statements without skips or repeats
>>> name = input('Please enter your name: ')
Please enter your name: Narendra

>>> program = input('Please enter your program: ')
Please enter your program: Game - Programming

>>> print(f'Hello {name} welcome to {program}')
Hello Narendra welcome to Game - Programming
```

The above code will always do the same thing: prompt for a string, read the string prompt for a second string and read that also. Then it prints a message. Programs written with only sequence control structure is capable of completing only a single task.

3.b – Conditionals

This control structure allows you to process or ignore code blocks depending (true/false blocks). This gives immense power to the language, now your programs are able to complete multiple/different tasks. Simply implement the individual tasks in different blocks. If there are more tasks to be implement then nest more conditionals.

Like most other modern languages there is a match statement that does the same job as a switch statement in other languages.

Notice unlike C# there is no parenthesis around the assertion nor are there curly braces to delimit the true or the else code blocks. Blocks are defined by indentations. Most IDE's will manage indentations for you (VS Code use 4 spaces for an indent), but if you paste code from another source, then there could be problems. Visually you may not be able to discern between 4 spaces and a tab, the Python interpreter will complain most vehemently.

3.b.1 – if

This is the basic syntax to test if a condition is true. If so, the indented code will be executed, if not it will be skipped.

```
n = 5
if n > 2:
    print('Larger than 2')
```

3.b.2 – else

This is a way to specify some code that will be executed if a condition is false.

```
age = 18
if age > 16:
    print('Can drive')
else:
    print("Don't drive")
```

3.b.3 – elif

In addition to the initial If statement condition, you can add extra conditions to test if the first condition is false.

Once an elif condition is true, the rest of the elif conditions are no longer checked and are skipped.

```
weather = 'sunny'
if weather == 'rain':
    print('bring umbrella')
elif weather == 'sunny':
    print('bring sunglasses')
elif weather == 'snow':
    print('bring gloves')
else:
    print('bring everything')
```

3.b.4 – and

This expects the conditions on both sides of the and to be true.

```
s = 58
if s < 60 and s > 50:
    print('Your grade is C')
```

3.b.5 – or

This expects either of the conditions on both sides of the or to be true. Basically, both conditions cannot be false.

```
age = 12
if age < 16 or age > 200:
    print("Can't drive")
```

3.b.6 – not

This will flip the original result of the condition. e.g. if it was true then it's now false.

```
if not 3 > 1:
    print('something')
#Will not be printed.
```

3.b.7 – Ternary statements

The *ternary statement* is a short hand if-else statement.

```
#ternary statement
«value if assertion is true» «assertion» «value if assertion is false»:

status = 'Wealthy' if asset > 1_000_000 else 'Normal'
```

There is a short hand version

```
#ternary statement
>>> a
345
>>> a or 'Hello'
345

>>> a and 'Hello'
'Hello'
```

Best practice:

- **Use only for simple choices**
- Avoid nesting more than once — prefer `if` block when readability suffers
- Never use the confusing `x or y / x and y` trick as a substitute — it is error-prone

3.c – Comparison Operators

These comparison (relational) operators allow you to compare two similarly typed values and either return `True` or `False`. They work the same way as in other languages. Python compares string **lexicographically** and list **element-by-element** (first difference decides).

3.c.1 – Greater than (`>`) operator

This operator will return `True` if the left side is greater than the right side otherwise it will return `False`.

```
>>> -2 > 3
```

```
False

>>> 1 > 1
False

>>> 3 > 2
True

>>> 1.2 > 1.4
False

>>> 3.1 > 3.1
False

>>> 9.8 > 7.2
True

>>> False > True
False

>>> True > True
False

>>> True > False
True

>>> 'apple' > 'zebra'
False

>>> 'zebra' > 'apple'
True

>>> 'baby' > 'baby'
False

>>> [4, 9] > [5, 0]
False
```

```
>>> [2, 3] > [2, 3]
False
```

```
>>> [5] > [4]
True
```

3.c.2 – Less than (<) operator

This operator will return True if the left side is less than the right side otherwise it will return False.

```
>>> -2 < 3
True
```

```
>>> 1 < 1
False
```

```
>>> 1.2 < 1.4
True
```

```
>>> False < True
True
```

```
>>> 'apple' < 'zebra'
True
```

```
>>> 'baby' < 'baby'
False
```

```
>>> [4, 9] < [5, 0]
True
```

3.c.3 – Greater than or equal (>=) operator

This operator will return True if the left side is greater than or equal to the right side otherwise it will return False.

```
>>> 3 >= 2
True

>>> 1.2 >= 1.4
False

>>> 3.1 >= 3.1
True

>>> 9.8 >= 7.2
True
```

3.c.4 – Less than or equal (<=) operator

This operator will return True if the left side is less than or equal right side otherwise it will return False.

```
>>> -2 <= 3
True

>>> 1.2 <= 1.4
True

>>> False <= True
True

>>> 'apple' <= 'zebra'
True

>>> [4, 9] <= [5, 0]
True
```

3.c.5 – Equality (==) operator

This operator will return True only if the left side is the same as the right side otherwise it will return False.

```
>>> 1 == 1
True

>>> 3.1 == 3.1
True

>>> 9.8 == 7.2
False

>>> False == False
True

>>> True == True
True

>>> 'baby' == 'baby'
True

>>> [2, 3] == [2, 3]
True
```

3.c.6 – Not equals (!=) operator

Return True if the left and right side values are not equal.

```
>>> -2 != 3
True

>>> 1 != 1
False

>>> 9.8 != 7.2
True

>>> True != False
True
```

```
>>> 'zebra' != 'apple'
True

>>> 'baby' != 'baby'
False

>>> [5] != [4]
True
```

3.d – Logic Operators

These operators allow you to implement more complex logic. These operators either take the first argument or the second argument. The “and” and the “or” operators take two arguments while the “not” operator takes a single expression.

Fun fact

These operators behave very differently from most other languages.

3.d.1 – Logical ‘and’ operator

The expression “x and y” first evaluates *x*; if *x* is false, its value is returned; otherwise, *y* is evaluated and the resulting value is returned.

```
# and returns true if both expressions are true, false otherwise

>>> age = 20
>>> province = "QC"

>>> age >= 18 and province == "ON"
False
```

3.d.2 – Logical 'or' operator

The expression "x or y" first evaluates *x*; if *x* is true, its value is returned; otherwise, *y* is evaluated and the resulting value is returned.

```
# or returns true if either expression is true, false otherwise
>>> age = 50
>>> service = 6

>>> age > 65 or service > 10
False
```

3.d.3 – Logical 'not' operator

The operator "not" yields "True" if its argument is false, "False" otherwise. Thus, this operator takes a single argument and return either True or False.

```
# not flips the bool value

>>> age = 50

>>> not age > 65
True
```

3.d.3.1 – Bit fiddling (This sub-section covers an advanced topic. You can safely ignore it.)

Bit fiddling (also known as bit manipulation or bit twiddling) in Python refers to the use of **bitwise operators** to directly manipulate the individual binary digits (bits) of an integer. This technique allows for efficient storage and manipulation of data at a low level, which is useful in specific scenarios like networking, cryptography, and embedded systems programming.

3.d.3.1.1 – Bitwise Operators

The bitwise operators are (&, |, ^, ~, << and >>) These operators take two integers and returns an integer. It treats numbers as a sequence of binary digits (bits) and perform operations bit by bit.

To get a better understanding binary values are used in the demonstration.

Please be advised these operators will only work with operands of the int type.

3.d.3.1.2 – Bitwise AND (&) Operator

The bitwise AND operator compares each bit of the first operand to the corresponding bit of the second operand. If both bits are 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

```
>>> 9 & 3      # and operator
1              # 1001 & 0011 = 0001

>>> 15 & 3
3              # 1111 & 0011 = 0011
```

3.d.3.1.3 – Bitwise OR (|) Operator

The bitwise OR operator compares each bit of the first operand to the corresponding bit of the second operand. If either bit is 1, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

```
>>> 9 | 3      # or operator
11             # 1001 | 0011 = 1011

>>> 15 | 3
15             # 1111 | 0011 = 1111
```

3.d.3.1.4 – Bitwise XOR (^) Operator

The bitwise XOR operator compares each bit of the first operand to the corresponding bit of the second operand. If the bits are not the same, the corresponding result bit is set to 1. Otherwise, the corresponding result bit is set to 0.

```
>>> 9 ^ 3      # xor operator
10             # 1001 ^ 0011 = 1010

>>> 15 ^ 3
12             # 1111 ^ 0011 = 1100
```

3.d.3.1.5 – Bitwise NOT (~) Operator

The bitwise NOT operator is a unary operator that flips the bits of its operand . Python uses **two's complement**, ($\sim x = -x - 1$) not simple bit flipping with fixed width).

```
>>> ~3         # not operator
-4             # result is based on two's complement representation
               # ~0b11 = 100
```

3.d.3.1.6 – Left Shift (<<) Operator

The left shift operator shifts the bits of the number to the left and fills 0 on voids left as a result.

```
>>> 9 << 1     # left shift operator
18             # 1001 <= 2 = 10010 (binary)

>>> 9 << 2
36             # 1001 <= 4 = 100100 (binary)
```

3.d.3.1.7 – Right Shift (>>) Operator

The right shift operator shifts the bits of the number to the right and fills 0 on voids left as a result.

```
>>> 9 >> 1    # right shift operator
4             # 1001 => 2 = 100 (binary)

>>> 9 >> 2
2             # 1001 => 4 = 10 (binary)
```

3.d.4 – Best Practice

- Use bitwise operators sparingly and only when necessary
- Avoid using bitwise operators to perform arithmetic operations, as this can make your code harder to understand.

3.e – Match statement

The `if` statement allows only two-way branching. In the situation where multi-way branching is required, then the `match` statement can be used.

This was introduced in Python 3.10 and is similar in purpose and working as the `switch` statement is most other languages.

```
day = input('Enter the three letter day of the week: ')

match(day.lower()):
    case 'sat' | 'sun':          #make it lowercase
                                #two cases combined using
        print(f'{day} is a family day')
    case _:                      #this will catch the remaining cases
        print(f'{day} is a work day')
```

```
x = int(input('Enter a number between 1 and 20: '))

match x:
    case 1:
        print('one')
    case 2 | 3:
        print('two or three')
    case x if x % 5 == 0:
        print('multiple of 5')
    case x if x > 16:
        print('greater than 16')
    case _:
        print('Value not in range')
```

```
name = input('Enter all your names: ')
names = name.split()
match names:
    case [x, y]:
        print(f'two names {x} {y}')
    case [x, y, z]:
        print(f'three names {x} {y} {z}')
    case [x, *y, z]:
        print(f"more than three names {x} {' '.join(y)} {z}")
    case _:
        print('Unkown error')
```

`match` is often more readable than long `if-elif` chains, especially when checking structure statement is more readable than the corresponding `if`-statement and in some (advanced) cases it is more powerful.

3.f – Summary

This chapter introduced **control structures**, the constructs that determine the order in which a Python program executes. Without them, a program can only run top-to-bottom and perform a single task; with them, programs can make decisions, repeat work, and respond to different inputs.

Key takeaways:

- **Sequential execution** is the default: statements run once, in order, with no skips. It is the foundation of all programs but is limited to a single, fixed behavior.
- **Conditionals** (`if`, `elif`, `else`) allow branching based on true/false conditions. Blocks are defined by indentation, not braces. The `elif` chain stops at the first true condition.
- **Logical operators** (`and`, `or`, `not`) build compound conditions. `and` and `or` use short-circuit evaluation and return one of their operands; `not` always returns a bool.
- **Comparison operators** (`>`, `<`, `>=`, `<=`, `==`, `!=`) compare values and return `True` or `False`. They work on numbers, strings (lexicographically), booleans, and lists (element-by-element).
- **Ternary expressions** (`value_if_true if condition else value_if_false`) provide compact one-line branching, but should be used sparingly for readability.
- **Bitwise operators** (`&`, `|`, `^`, `~`, `<<`, `>>`) manipulate integers at the binary level. They are powerful but specialized — best reserved for low-level tasks like networking, cryptography, and embedded systems.
- **The `match` statement** (Python 3.10+) enables clean multi-way branching, similar to `switch` in other languages. It supports literal patterns, OR patterns (`|`), guards (`case x if ...`), and structural patterns for sequences.
- **Loop-altering keywords** (`break`, `continue`, `return`) modify normal flow inside control structures.

3.g – Non coding exercises

1. Which control structure guarantees each statement runs exactly once, in order?
2. What does `elif` do?
3. Which operator returns `True` only when both sides are true?
4. What does `~3` evaluate to in Python?

5. Which Python version introduced the `match` statement?
6. What does the `break` keyword do?
7. In Python, code blocks are delimited by curly braces `{ }`. (True or false?)
8. `and` and `or` always return a boolean value. (True or false?)
9. `1.2 <= 1.4` evaluates to `True`. (True or false?)
10. Bitwise operators work on floats as well as ints. (True or false?)
11. `match` can replace long `if-elif` chains and is often more readable. (True or false?)
12. `not` is a binary operator. (True or false?)
13. What is the result of `5 > 3 and 2 > 4`?
14. What is the result of `not (3 == 3)`?
15. What is the result of `'apple' < 'banana'`?
16. What is the result of `[1, 2] < [1, 3]`?
17. What is the result of `9 & 5` (in binary: `1001 & 0101`)?
18. What is the result of `9 | 5`?
19. What is the result of `9 ^ 5`?
20. What is the result of `4 << 2`?
21. What is the result of `20 >> 2`?
22. Why sequential-only programs are limited?
23. Why does Python rely on indentation instead of braces? What risk does this create when pasting code?
24. Give an example of a real-world decision that maps naturally to an `if-elif-else` chain.
25. When would you choose `match` over `if-elif`? Give one scenario.
26. Why should bitwise operators be used sparingly?
27. What is short-circuit evaluation, and why is it useful?

4 – REPETITION

Repetition structures allow a block of code to run zero or more times. Without loops, many practical programs would be inefficient or outright impractical – for example, adding the numbers from one to one million. Loops are essential for processing collections, performing repeated calculations and automating repetitive tasks.

Python supports two main looping constructs:

- while loops
- for loops

Python does **not** support:

- do-while loops
- C-style for(init; condition; update) loops

Both loop types support the `break` and `continue` keywords.

4.a – While statements

The while loop performs **conditional iteration**. The statements in the loop body are processed **as long as** the condition is `True` and processing ceases when the assertion becomes `False`. It is useful when the number of iterations is not known in advance.

```
while «condition»:  
... # body of the loop
```

If the condition never becomes `False`, you will have created an infinite loop. Always ensure your loop has logic to eventually terminate.

```
>>> x = 1  
>>> total = 0  
>>> while x < 100:  
...     total += x
```

```
... x += 1
...
>>> print(total)
4950
```

4.b – For loop

A for loop iterates over the elements of an iterable, so the number of iterations matches the number of elements produced.

Common Iterables:

- strings
- lists
- tuples
- sets
- dictionaries
- range() object

4.b.1 – Iterating Over an Iterable

Iterables include strings, lists, sets, tuples and dictionaries.

```
# list example
>>>a = [3, 5, 7, 9]
>>>for x in a:
...   print(x, end=' ')
...
3 5 7 9 >>>
```

```
# string example
>>> name = 'Narendra'
>>> for a in name:
...   print(a, end=' ')
...
```

```
N a r e n d r a >>>
```

```
fruits = ['apple', 'banana', 'orange']
for f in fruits:          #fruits is a list
    print(f)
```

4.b.2 – Using the range object

The range() function returns an immutable sequence of numbers. A for loop is normally used to iterate this sequence. The range function takes one mandatory and two optional arguments.

```
range(stop)
range(start, stop)
range(start, stop, step)
```

```
>>>for x in range(3, 10, 2):
...   print(x, end=', ')
...
3, 5, 7, 9 >>>
```

4.b.3 – Using the _ (Discard Variable)

If the loop variable is not needed, use the _. This avoids linter warning and clearly signals intent.

```
# if you are not using the value, then the linter
```

```
# will complain about un-used variables. The under
# score (referred to as a 'discard') prevents this
# often annoying reporting.
>>>for _ in range( 10 ):
...   print('Hello', end=' ')
Hello Hello Hello Hello Hello Hello Hello Hello Hello Hello
```

4.b.4 – Using the Enumerate Function

The `enumerate()` function returns index-value pairs from a collection (a sequence or mapping).

```
enumerate(«collection», [«start=0»])
```

```
>>> profs='ilia yin arben narendra mehrdad'.split()
>>> #process all the items
... for index, item in enumerate(profs):
...   print(f'{index} {item}')
...
0 ilia
1 yin
2 arben
3 narendra
4 mehrdad
```

```
# you can start the count at a particular integer
>>> profs='ilia yin arben narendra mehrdad'.split()
>>> for index, item in enumerate(profs, start=2):
...   print(f'{index} {item}')
...
```

```
2 ilia
3 yin
4 arben
5 narendra
6 mehrdad
```

```
# you can iterate a dictionary
>>> profs = {'ilia': 'COMP304', 'yin': 'COMP306', 'arben': 'COMP305',
            'narendra': 'COMP216', 'mehrdad': 'COMP225'}
>>> for index, key in enumerate(profs):
...     print(f'{index}: {key}')
...
0: ilia
1: yin
2: arben
3: narendra
4: mehrdad
```

```
# a better way to iterate a dictionary is to use the .items() method of the
dictionary class
>>>
>>> for prof, course in profs.items():
...     print(f'{prof.rjust(12)}: {course}')
...
      ilia: COMP304
      yin: COMP306
      arben: COMP305
narendra: COMP216
mehrdad: COMP225
```

4.b.5 – Nested Loop

A loop within another loop is referred to as a nested loop. The two loops (inner and outer) do not need to match. It is possible to have multiple-level of nested loops.

```
distros = 'ubuntu mint centos'.split()
arch = '32-bit 64-bit'.split()
for d in distros:
    for a in arch:
        print(d, a)
        print('_____')
```

ubuntu 32-bit

ubuntu 64-bit

mint 32-bit

mint 64-bit

centos 32-bit

centos 64-bit

4.c – Loop control Keywords

4.c.1 – break

This keyword allows you to break free of the loop. You can use it in a for or while loop to terminate when a particular condition is detected.

```
scores = [34, 67, 99, 105]
for s in scores:
    if s > 100:
        print('Invalid')
        break
    print(s)
```

4.c.2 – continue

This keyword allows you to skip the rest of the statement in the loop body and go to the next. The loop will still continue, but it will start from the top of the loop statement.

```
n = 0
while n < 100:
    n += 1
    if n % 2 == 0:
        continue
    print(n)
#Prints all the odd numbers
```

4.c.3 – pass

This keyword allows you to place a nothing statement (one that does not do anything) where a statement is needed. It is often used when you are not sure or you have not decided on the implementation details. It can be used for the body of a class, a method, the body of a loop or a block in an if statement.

```
for x in range(20):
    if x % 3 == 0:
        pass          #a nothing statement
    else:
        print(x)
```

4.d – Infinite Loops

Sometimes, the condition you are checking to see if the loop should continue never becomes false. In this case, the loop will continue for eternity (or until your computer stops it). This is more common with while loops.

```
# you may stop the following loop by pressing ctrl+c
while 5 > 1:
    print("I'm a survivor")
    input('')
```

4.e – Summary

This chapter introduced repetition structures, which allow a block of code to execute zero or more times. Loops are essential for processing collections, automating repetitive tasks, and writing efficient programs — without them, tasks like summing a million numbers would be impractical.

Key takeaways:

- Two loop constructs in Python:
- `while` — conditional iteration; repeats as long as a condition is `True`. Best when the number of iterations is not known in advance.
- `for` — determinate iteration; iterates over the elements of an iterable (strings, lists, tuples, sets, dictionaries, `range()` objects, etc.).
- Python does not support do-while loops or C-style `for(init; condition; update)` loops.
- `range()` generates an immutable sequence of numbers. It supports one, two, or three arguments: `range(stop)`, `range(start, stop)`, and `range(start, stop, step)`.
- `enumerate()` yields index-value pairs, useful for tracking position while iterating. It accepts an optional start argument.
- `zip()` (not covered in the original but worth knowing) iterates multiple iterables in parallel.
- **Nested loops** place one loop inside another; the inner loop runs completely for each iteration of the outer loop.
- **Loop control keywords:**
 - `break` — exits the loop immediately.
 - `continue` — skips the rest of the current iteration and moves to the next.
 - `pass` — a no-op placeholder, used when a statement is syntactically required but no action is needed yet.
- **Infinite loops** occur when the loop condition never becomes `False`. Always ensure termination logic; use `Ctrl+C` to interrupt a runaway loop. A common intentional pattern is `while True:` with a `break` inside.
- Loop else clause (a Python-specific feature): the `else` block after a `for` or `while` runs only if the loop finished without hitting a `break`.

In short: while loops are condition-driven, for loops are collection-driven, and the combination of break, continue, and pass gives fine-grained control over loop behavior. Proper termination is critical to avoid infinite loops.

4.f – Non coding exercises

1. Which loop is best when the number of iterations is not known in advance?
2. What does `range(2, 10, 3)` produce?
3. What does `enumerate(['a', 'b'])` yield?
4. Which keyword exits a loop immediately?
5. Which keyword does nothing and is used as a placeholder?
6. How many arguments can `range()` accept?
7. What does the else clause on a for loop do?
8. Python supports do-while loops. (True or false?)
9. A while loop always runs at least once. (True or false?)
10. `break` and `continue` can be used in both for and while loops. (True or false?)
11. `range(5)` produces the numbers 1 through 5. (True or false?)
12. `enumerate()` can start counting at a custom number. (True or false?)
13. A nested loop's inner loop runs once per outer-loop iteration. (True or false?)
14. `pass` terminates the loop. (True or false?)
15. What the difference between a while loop and a for loop. Give a scenario where each is the better choice.
16. Why does Python use `_` as a loop variable? What problem does it solve?
17. What is an infinite loop? How can you avoid one? How can you stop one that is already running?
18. When would you use `enumerate()` instead of a plain for loop?
19. What is a nested loop? Give a real-world example where nesting is natural.
20. Why might you use `while True:` with a `break` instead of a normal while condition?
21. Explain the difference between `break`, `continue`, and `pass`.

5 – FUNCTIONS

Functions are a fundamental control structure that allow you to give a name (*a label*) to a reusable block of code. Once defined, that block can be reused anywhere in your program simply by calling its name. This promotes code reuse, modularity, reduces duplication, and lowers the chance of errors.

You can think of a function as a small program with a **single responsibility**. A function may:

- accept inputs (called *parameters* or *arguments*),
- perform some logic, and
- optionally return one or more output values.

Functions can call other functions, and in some cases, even call themselves (a technique known as *recursion*).

Using functions naturally encourages **problem decomposition**—breaking a complex problem into smaller, simpler tasks. Each task is easier to implement, reason about, test, and maintain.

Python supports:

- **default (optional) arguments**
- **keyword arguments**
- **arbitrary arguments** (a variable number of arguments)
- **anonymous functions** (functions without names, usually created with `lambda`)

Unlike most modern languages (JavaScript is a notable exception), python does not support function overloading i.e. multiple functions with the same name. If you define multiple functions with the same name, only the last definition remains active. You can partially simulate function overloading by using default arguments.

You may safely skip the section on Higher-order functions and Closures (section 1.c.4.3 and 1.c.4.4).

5.a – User-Defined Functions

5.a.1 – Creating Functions

Functions are defined using the `def` keyword, followed by:

1. the function name,
2. parentheses containing optional parameters
3. a colon, and
4. an indented block that forms the function body.

```
def foo():  
    print('Programming 1 is the best course')
```

Indentation is mandatory and tells Python which statements is part of the function.

5.a.2 – Calling Functions

To execute a function, write its name followed by parentheses.

```
foo()  
foo()  
#prints the message twice
```

5.a.3 – Functions with Inputs

Functions become far more useful when they accept inputs. Each call can then produce different result depending on the provided arguments.

```
def add(first, second):  
    print(first + second)  
  
add(2, 3)           #will output 5  
add(5, 7)           #will output 12
```

5.a.4 – Functions that explicitly return a value

A function can return a value using the `return` keyword. Returned values can be captured in variables or used in expressions.

Strange fact

Return value of a function is not the same as what is printed to the screen.

```
def calculate_tax(price):  
    return price * 0.15
```

```
tax = calculate_tax(2)    #tax is calculated on $2.00
```

Note: Functions without an explicit return statement automatically returns None.

```
def foo():  
    a = 1 + 3  
  
print(foo())    # outputs None
```

5.a.5 – Variable Scope

Variables defined inside a function exist only while that function is executing. Where a variable is defined determines *where it can be accessed*.

```
a = b = c = 1    #global scope  
def foo():  
    global a  
    a, b = 2, 2    #b is local  
    print(f'{a} {b} {c}')  
  
foo()    #Prints 2 2 1  
print(f'{a} {b} {c}')    #Prints 2 1 1
```

Use the `global` keyword sparingly—it can make programs harder to reason about.

5.a.6 – Keyword Arguments

When calling a function, you can provide a keyword argument or simply just the value. Using a keyword argument means that you don't have to follow any order when providing the inputs.

```
def divide(dividend, divisor):  
    result = dividend / divisor  
  
divide(10, 5)    #Option 1:  
divide(divisor =5, dividend =10) #Option 2:
```

5.a.7 – Default Arguments

If the value of one of the arguments to a function is normally unchanged, then you can simplify the calling of the function by setting a default value for that particular argument. The `split()` and `print()` methods have default argument of ' ' and '\n' respectively.

```
def greet(name, greeting = 'Hi'):
    print(f'{greeting} {name}')

greet('Ilia')           #output: Hi Ilia
greet('Hao', 'Hello')   #output: Hello Hao
```

5.a.8 – Annotation

Python support function annotations for both parameter and return type. It provides additional information about a function's argument and return type. This metadata is specified using a colon (:) for parameters and an arrow (->) for the return type. This information is stored in the function's `__annotations__` attribute.

This is completely optional; python does not attach any meaning or significance to it. However, it can be used by third-party applications to enhance their operations. IDE such as Visual Studio Code can provide type-checking and code hinting. It can be also be used for foreign-language bridges, predicate logic functions database query mappings etc.

```
def «function_name»(«list_of_argument(s)»):
    Body of function
```

```
>>> def hello(name: str) -> None:
...     print(f'Hello world, from {name}')
...
>>>
>>> hello('ilia')
```

```
Hello world, from ilia
```

The above example demonstrates “type hinting”, and it makes your code more user friendly. Although type hinting is optional, we will try to use it as much as possible in this course.

If a function does not explicitly have a return statement, then a None type is returned.

```
>>> def foo( ):
...     a = 1 + 3
...
>>>
>>> print(foo())
None
```

5.a.9 – Recursive Functions

A function that calls itself is a recursive function. A popular example of recursion is the factorial function which is defined by the following two statements:

$$F_1 = 1$$

$$F_n = n * F_{n-1}$$

```
def fact(n: int) -> int:      #annotations
    if n == 1:
        return 1
    else:
        return n * fact(n-1)

n = 5
print(f'The factorial of {n} is {fact(n)}')
```

Another example of a recursive function is the Fibonacci function which is defined by the following three statements:

$$F_0 = 0$$

$$F_1 = 1$$

$$F_n = F_{n-2} + F_{n-1}$$

Challenge

Can you implement the factorial or the Fibonacci functions without recursion?

```
def fib(n: int) -> int:
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n-2) + fib(n-1)
n = 8
print(f'The {n}th fibonacci number is {fib(n)}')
```

Recursion provides elegant implementation for lots of classical problems in computer science such as searching and sorting.

5.a.10 – Inner Functions

A function that is declared within another function is an inner function or a nested function. The inner function has access to the variable of the enclosing function so it is often used as helper functions. It can also be used to provide encapsulation or hiding the inner function. Unless the inner function is extremely short it is a better practice to use private helper functions. When deciding to use inner function, you should consider code readability and maintainability. You might think of using lambda expression instead.

Another popular use of inner function is closures that is covered in the part of this section.

Inner functions are useful for encapsulation and closures, but avoid deeply nested or overly long inner functions — prefer module-level private helpers (`_name`) when readability suffers.

```
def talk(phrase: str) -> None:
    def say(word: str) -> None:      #inner function
        print(word)

    words = phrase.split()
    for word in words:
        say(word)

>> talk('COMP216 is the best course')

COMP216
is
the
best
course
```

5.a.11 – Higher-order Functions

In Python functions are first-class objects. They can be stored as variables, passed as arguments and returned from other functions.

```
def shout(text: str) -> str:      #annotations
    return text.upper()

def whisper(text: str) -> str:
    return text.lower()

txt = 'Hi Narendra'
shout(txt)                        #HI NARENDRA
```

```
yell = shout                #assigns function to a variable
yell(txt)                   #HI NARENDRA

whisper(txt)                #hi narendra

def greet(func) -> str:     #function that takes a function argument
    return func('from a higher power')

print(greet(shout))         #FROM A HIGHER POWER

print(greet(whisper))       #from a higher power

...

A function that returns a function.
...

def make_entity(kind) -> (str):
    def client():           #a nested function definition
        return 'I am a client'

    def server():
        return 'I am a server'

    if kind == 'client':
        return client
    else:
        return server

kind = 'client'
f = make_entity(kind)
print(f())                  #I am a client

kind = 'some_thing'
f = make_entity(kind)
print(f())                  #I am a server
```

5.a.12 – Closures

A closure is a function that remembers values from its enclosing scope even after that scope has finished executing.

A closure must satisfy the following:

- There should be nested function i.e. function inside a function.
- The inner function must refer to a non-local variable or the local variable of the outer function.
- The outer function must return the inner function.

```
...
A function that returns a closure.
...

def adder(x) -> (int):
    def add(y) -> int:
        return x + y

    return add                                #returns a closure of the captured x

incrementBy2 = adder(2)                       #a closure that adds 2
incrementBy3 = adder(3)                       #a closure that adds 3
incrementBy9 = adder(9)                       #a closure that adds 9

print(incrementBy2(3))                        #5
print(incrementBy3(7))                        #10
print(incrementBy9(30))                       #39
```

~~5.a.12.1 – Lambda Based Closures~~

~~A closure is a function object (a function that behaves like an object) that remembers values in enclosing scopes (variables etc.) even if they are not present in~~

~~memory. An inner function is able to access the (non-local) variable of the outer function.~~

~~A closure must satisfy the following:~~

- ~~• There should be nested function i.e. function inside a function.~~
- ~~• The inner function must refer to a non-local variable or the local variable of the outer function.~~
- ~~• The outer function must return the inner function.~~

```
'''  
A function that returns a closure.  
'''
```

5.b – Documentation

Explains the purpose and function of the code.

```
#This function calculates the area of a rectangle  
def calculate_area(length, width):  
    return length * width
```

5.b.1 – Code Explanation

To provide clarity on complex logic or non-obvious code segments or workarounds.

```
#Using list comprehension to get the multiples of 3 less than 20  
multiple = [x for x in range(20) if x % 3 == 0]
```

5.b.2 – Disabling Code

To temporarily disable or comment out a section of code for testing or debugging.

```
#print('This line will not be processed by the interpreter')
```

5.b.3 – To-Do Notes

To mark section of code that need further attention such as pending tasks or later improvements.

```
#TODO: Implement error handling for invalid input
def divide(a, b):
    return a / b
```

5.b.4 – Code Maintenance

To help developer in the future understand the code history and rationale.

```
# This workaround is necessary due to a known bug in the library
if library_version < '1.0.0':
    # Use the old method
    result = old_method()
else:
    # Use the new method
    result = new_method()
```

5.b.5 – Code Review

To add notes during code review processes, helping reviewers understand specific decisions or changes.

```
# Reviewer note: This check is necessary to prevent division by zero
if denominator != 0:
    result = numerator / denominator
else:
    result = 0
```

5.b.6 – Debugging Information

To leave notes that might help in debugging, such as variable values or conditions.

```
# Debug: Checking the value of variable x
```

```
x = 10
# print(f"x is {x}")
```

5.b.7 – Best Practices for Writing Comments

Be Clear and Concise: Write comments that are easy to understand and to the point.

Update Regularly: Keep comments up-to-date as the code changes. Outdated comments can confuse readers and lead to bugs.

Avoid Redundant Comments: Don't comment on obvious code. Instead, focus on explaining complex logic or non-obvious decisions. Do not explain how but why.

Use Consistent Style: Follow a consistent style for comments to maintain readability.

Use Docstring for Documentation: Use docstrings for modules, classes and functions to describe their purpose, parameters and return values. (See the appendix for a more thorough coverage.)

By using comments effectively, you can make your code more maintainable, understandable, and easier to debug.

5.c – Built-in Functions

These are functions that are available in all instances of python without you explicitly coding it. They are very useful for both the novice and the expert programmer. Some of the more useful ones are:

- **id(x)**
returns the id of the argument. In cpython (the normal implementation) it is normally the object's memory address.
- **type(x)**
outputs the underlying type of the argument.
- **del(x)**
removes the binding of a name from the current environment.

- **dir(x)**
outputs all the members (data as well as functional attributes) of the argument.

If this function is invoked without any arguments, it will list all the variables in the current environment.
- **help(x)**
outputs same the same as the above with additional explanation on each member.
- **print(x)**
outputs the stringified version of the argument.
- **isinstance(obj, type)**
returns a bool indicating if the first argument is of type of the second argument.
- **hasattr(obj, attr)**
returns a bool indicating if the first argument has a member of type of the second argument. The second argument is of type string.
- **getattr(obj, attr)**
returns the value of the attribute specified by the second argument. If the attribute does not exist on the object then an exception is raised.
- **dir(__builtins__)**
returns a list of all the built-in function available in the default environment of the current python distribution.

Some other useful functions are: **str()**, **int()**, **float()**, **abs()**, **round()**, **max()**, **min()**, **len()**, **pow()**, **ord()**, **strip()**, **upper()**, and many, many more.

1.e – Scope

Scoping refers to the rules that determine where a particular variable is accessible and what value it holds. Understanding these rules is essential for writing correct and maintainable code. The python interpreter follows LEGB (Local -> Enclosing -> Global -> Builtin) when executing a program.

1.e.1 – Local Scope

This is the innermost scope, typically associated with a function or method.

```
def foo():  
    a = 5  
    print(f'a: {a}')  
print(f'a: {a}')
```

 #a in not accessible here

1.e.2 – Enclosing Scope

This deals with nested functions.

```
def outer():  
    a = 5  
  
    def inner():  
        print(f'a: {a}')
```

 #a is in an enclosing scope

 inner()

outer()

1.e.3 – Global Scope

Global scope in the outermost scope of a program. To modify a global variable you must use the global keyword.

```
CURRENT_ID = 1_000
```

```
def generate_id ():  
    global CURRENT_ID      #the global keyword is necessary here  
    CURRENT_ID += 1
```

1.e.4 – Built-in Scope

This is the widest scope! All the special reserved keywords falls under this scope. These can be accessed directly without any imports.

Randomisation

The random functions come from the random module which needs to be imported. In this case, the start and end are both included:

start <= randint <= end

```
import random  
# randint(start, end)  
n = random.randint(2, 5)  
#n can be 2, 3, 4 or 5.
```

range

Often you will want to generate a range of numbers. You can specify the start, end and step. Start is included, but end is excluded:

start >= range < end

```
# range(start, end, step)  
for i in range(6, 0, -2):  
    print(i)          # displays 6, 4, 2 but not 0
```

round

This does a mathematical round. So 3.1 becomes 3, 4.5 becomes 5 and 5.8 becomes 6.

```
round(4.6)           #result 5
```

abs

This function work with numeric types, it returns the absolute value of its argument. Basically, removing any negative signs.

```
abs(-4.6)          #result 4.6
```

5.d – Summary

This chapter introduced **functions**, the control structure that lets you name and reuse a block of code. Functions are the primary tool for **problem decomposition** — breaking a complex problem into smaller, testable pieces — and they promote code reuse, modularity, and maintainability.

Key takeaways:

- **Defining and calling:** Use `def name(params):` to define a function and `name(args)` to call it. The body must be indented.
- **Parameters and arguments:** Functions can accept positional arguments, keyword arguments (order-independent), default arguments (optional), and arbitrary arguments (`*args`, `**kwargs`).
- **Return values:** `return` sends a value back to the caller. A function without an explicit `return` returns `None`. A function technically returns a single object — tuples give the appearance of multiple return values.
- **Scope and LEGB:** Python resolves names using Local → Enclosing → Global → Built-in lookup. Use the `global` keyword sparingly to modify globals; use `nonlocal` for enclosing scopes.
- **Annotations:** Optional type hints (`name: str, -> int`) document expected types. They are stored in `__annotations__` and used by IDEs and tools, but Python does not enforce them.
- **Recursion:** A function that calls itself. Classic examples: factorial and Fibonacci. Requires a base case to terminate.
- **Inner functions:** Functions defined inside other functions. Useful for encapsulation, helpers, and closures — but prefer module-level private helpers when readability suffers.
- **Higher-order functions:** Functions that take or return other functions. Python treats functions as first-class objects.
- **Closures:** A nested function that remembers variables from its enclosing scope even after the outer function returns. A closure requires (1) a nested

function, (2) a reference to an outer variable, and (3) the outer function returning the inner function.

- **Lambda:** Anonymous, single-expression functions (`lambda x: x + 1`). Useful for short callbacks.
- **Documentation:** Comments explain why; docstrings document what a module/class/function does. Keep comments clear, current, and non-redundant.
- **Built-in functions:** `id()`, `type()`, `dir()`, `help()`, `print()`, `isinstance()`, `hasattr()`, `getattr()`, `len()`, `abs()`, `round()`, `max()`, `min()`, etc. `del` is a **statement**, not a function. Watch out for `round()` using **banker's rounding** (round-half-to-even).

In short: functions let you package logic under a name, accept inputs, return outputs, and compose larger programs from small, focused pieces. Mastering scope, arguments, recursion, and closures is essential for writing clean, reusable Python.

5.e – Non-Coding Exercises

1. Which keyword defines a function?
2. What does a function return if it has no return statement?
3. Which is the correct order of Python's name lookup (LEGB)?
4. What does `*args` allow a function to accept?
5. Which keyword lets a function modify a global variable?
6. What is a closure?
7. What is `round(4.5)` in Python 3?
8. A function without a return statement returns `None`. (True or false?)
9. Python supports true function overloading by signature. (True or false?)
10. `global` should be used frequently for convenience. (True or false?)
11. Type annotations are enforced at runtime by Python. (True or false?)
12. A closure requires the outer function to return the inner function. (True or false?)
13. `range(5, 0, -1)` includes 5 and excludes 0. (True or false?)
14. `random.randint(1, 5)` can return 5. (True or false?)
15. `del(x)` deletes the object `x` refers to, not just the name binding. (True or false?)
16. What the difference between a parameter and an argument?
17. Why does Python return `None` from a function with no return?
18. What is the difference between `global` and `nonlocal`?
19. Why are type annotations optional, and what are they used for?
20. What is recursion? Why must every recursive function have a base case?

21. Explain what a closure is and give a practical use case.
22. When would you use a lambda instead of a def?
23. Why does the chapter recommend avoiding the global keyword?
24. What is the difference between a comment and a docstring?

6 – SEQUENCES

A sequence is an ordered collection of items that supports operations such as indexing, slicing, iteration and membership testing. They form a core part of the python language with many built-in types as sequences.

Sequences may be categorized into two main groups: Immutable and mutable.

Immutable

Immutable types include `str`, `tuple` and `bytes`. Variables of these types do not change after initialization. When a variable of these type is changed, the new value is assigned to a new location. The existing variable reference is changed to point to the new location.

Information

The numeric types `int` and `float` are immutable as well as the set type `frozenset`.

```
# Demonstrates immutability
a = 'hello'
b = a
print(f'{a:<20} {b}')
hello                hello

print(f'{id(a):<20} {id(b)}')
1389813655632        1389813655632

a = 'world'           # a new value is assigned to a
print(f'{a:<20} {b}')
world                hello

print(f'{id(a):<20} {id(b)}')
1389818780176        1389813655632
```

Mutableables

Mutable types include `list` and `bytearray`. and most custom type (types that are user defined). When a variable of these types is changed, the new value is assigned to the existing location.

Information

The mapping type `dict` is immutable as well as the set type `set`.

```
# Demonstrates mutability
a = [1]
b = a
print(f'{a} {b}')
[1] [1]

print(f'{id(a):<20} {id(b)}')
1389824700992      1389824700992

a.append(2)
print(f'{a} {b}')
[1, 2] [1, 2]

print(f'{id(a):<20} {id(b)}')
1389824700992      1389824700992
```

6.a – String

A string is a sequence of characters that is delimited with either a pair of single quotes, double quotes or triple single or triple double quotes. In a string literal, you may not use mismatch quotes as your delimiter. i.e. the start and end delimiters must be identical in a literal definition.

Strange fact

Python does not have a native `char` data type as does most modern languages.

```
a = 'Hello '  
b = "world"  
c = '''This string  
can span multiple  
lines.'''
```

6.a.1 – String Creation Methods

The `str()` function can create a string from pretty much any literal. There are also lots of functions that return a string.

```
>>> str('Hello World')  
'Hello World'  
  
>>> str(3.14159)  
'3.14159'  
  
>>> str([1, 2])  
'[1, 2]'  
  
>>> str(True)  
'True'  
  
>>> str(None)  
'None'
```

6.a.1 – String Concatenation

You can add strings to strings to create a new string. This is called concatenation and is supported by most modern high-level languages. It results in a new string. In Python you are able to multiply a string by an integer.

Fun fact

Ruby and PERL also allow you to multiply a string by an integer.

```
>>> 'Hello ' + 'Narendra'  
'Hello Narendra'
```

```
>>> 'Hi ' * 3
'Hi Hi Hi '
```

6.a.2 – Escaping a String

Because quotations are special, (it denotes the start and end of a string) if you want to use a quote in a string, you need to escape it with a `'\'`

```
>>> 'She said \'hi\''
"She said 'hi'"

>>> print('Hi\nNarendra')           #embedding a newline character
Hi
Narendra

>>> print('Hi\tNarendra')           #embedding a tab
Hi      Narendra
```

6.a.3 – Raw strings

All string in python are checked for meta-characters such as `\n` (newline) and `\t` (tab). As you may notice, these sequences are prefixed with the backslash character (`\`). If you want to use the backslash as part of your string, then you have to prefix it with another backslash. Thus, `print('\\/n')` with output `'/n'`. This can be tedious to type and hard to read, so the language designers came up with a technique that allows a programmer to indicate to the interpreter that a string should not be processed. Raw strings are especially convenient when using regular expressions.

Fun fact

Other languages that support raw strings are C#, C++ and Rust. Java does not support raw strings.

A prefix of `r` (or `R`) before a string tells python that the string should not be processed.

```
>>> print(r'\n')                    #raw string
\n
```

```
>>> print('\n')           #normal string

>>> print('\n\n')        #normal string
\n
>>> print(r'\n\n')       #raw string
\\n
>>>
```

6.a.4 – Combining F-Strings and raw strings

It is possible to embed an expression within a raw string as the example below shows:

```
>>> base = 'facultyweb.centennialcollege.ca'
>>> print(rf'http://{base}/npersh')
http://facultyweb.centennialcollege.ca/npersh
```

6.b – List

A list is a collection of values that can be of any type. It is mutable i.e., it is possible to modify an existing list. You may perform indexing and slicing on a list.

6.b.1 – Creating Lists

You may create list using the following statements:

```
a = list()
b = []
c = [9, 8, 7]
d = 'Mary had a little lamb'.split()
e = [0, 'one', [2], 3.0, True]
```

6.b.2 – Adding Lists Together

You can extend a list with another list by using the extend keyword, or the + symbol.

```
list1 = [1, 2, 3]
list2 = [9, 8, 7]
new_list = list1 + list2
list1 += list2
```

6.b.3 – Adding an Item to a List

If you just want to add a single item to a list, you need to use the `append()` method.

```
fruits = ['apple', 'banana', 'orange']
fruits.append('pear')
```

The `in` operator

This operator returns `True` if the item exists in the list.

```
colors = ['red', 'green', 'blue']
color = 'white'
print(f'{color in colors}')      #display False
```

List Indexing

To get hold of a particular item from a list you can use its index number. This number can also be negative, if you want to start counting from the end of the list.

```
letters = ['a', 'b', 'c']
letters[0]          #give 'a'
letters[-1]         #the last item 'c'
letters[0] = 'z'    #sets the first item to 'z'
```

List Slicing

Using the list index and the colon symbol you can slice up a list to get only the portion you want. Start is included, but end is not.

```
#list[start:end:step]
```

```
letters = ['a', 'b', 'c', 'd']  
letters[1:3]          #Result: ['b', 'c']
```

6.c – Common Sequence Operations

The following operations can be performed on all sequences.

6.b.1 – min(), max() sum()

These work only if all the items are numeric.

```
>>> max((8, 6, 4.6, -9))  
8  
>>> sum([8, 6, 4.6, -9])  
9.600000000000001  
>>> min((8, 6, 4.6, -9))  
-9
```

6.b.1 – Length

Returns the count of the number of first-level items in the sequence.

```
>>> len('Narendra')  
8  
>>> len((8, 6, 4.6, -9))  
4  
>>> len([8, [], 'hello'])  
3
```

6.b.1 – Membership test

The **in** operator is used to determine if an item is present in a list.

```
>>> print('N' in 'Narendra')
```

```
True
>>> print('E' in 'Narendra')
False
>>> print([] in [8, [], 'hello'])
True
>>> print(0 in [8, [], 'hello'])
False
```

6.b.1 – Concatenation

Concatenation or joining is accomplished with the plus operator. This is only supported between two identical types: str and str, tuples and tuples and list and list

```
>>> 'Narendra ' + 'Pershad'    # two strings
'Narendra Pershad'
>>> (1, 2) + (3, 4)             # two tuples
(1, 2, 3, 4)
>>> [1, 2] + [3, 4]             # two lists
[1, 2, 3, 4]
```

For set use the union (|) operator

```
>>> {1, 2} | {3, 4}
{1, 2, 3, 4}
```

For dict use the | operator

```
>>> {1: 'one'} | {2: 'two'}
{1: 'one', 2: 'two'}
```

6.b.1 – Repetition

```
>>> 'Narendra ' * 3           #string
'Narendra Narendra Narendra '
>>> (1, 2) * 2                #tuple
(1, 2, 1, 2)
>>> [1, 2] * 2                #list
[1, 2, 1, 2]
```

6.b.1 – Iteration

6.b.1 – Indexing

6.b.1 – Slicing

6.d – Summary

This chapter introduced **sequences**, ordered collections that support indexing, slicing, iteration, and membership testing. Python's built-in sequence types fall into two categories: **immutable** (str, tuple, bytes) and **mutable** (list, bytearray).

The chapter focused primarily on strings and lists, the two most commonly used sequence types.

Key takeaways:

- **Immutability vs. mutability:**
 - Immutable objects (`str`, `tuple`, `bytes`) cannot be changed after creation. Assigning a "new value" rebinds the name to a new object.
 - Mutable objects (`list`, `bytearray`, `set`, `dict`) can be modified in place; changes are visible through all references to the same object.
- **Strings:**
 - Created with single, double, or triple quotes; triple quotes allow multi-line strings.
 - Concatenation with `+`; repetition with `*`.
 - Escape sequences: `\n` (newline), `\t` (tab), `\'` (quote), `\\` (backslash).
 - Raw strings (`r' . . . '`) disable escape processing — useful for regex and Windows paths.
 - F-strings can be combined with raw strings (`rf'...'`).
- **Lists:**
 - Created with `list()`, `[]`, or `str.split()`.
 - Combined with `+` (new list) or `extend()` / `+=` (in place).
 - Single items added with `append()`.
 - Support indexing (including negative indices) and slicing (`list[start:end:step]`, end-exclusive).
- **Common sequence operations (work on all sequences):**
 - `min()`, `max()` — work on comparable items; `sum()` — numeric only.
 - `len()` — counts first-level items.
 - `in` — membership test.
 - `+` — concatenation (same type only).
 - `*` — repetition.
 - Iteration with `for`.
 - Indexing and slicing.
- **Merging collections:** sets and dicts use `|` for union/merge (not concatenation).
- **Note:** tuple, bytes, and bytearray were mentioned but not covered in depth; strings and lists were the primary focus.

In short: sequences are the workhorses of Python data handling. Understanding immutability, indexing, slicing, and the difference between in-place vs. new-object operations is essential for writing correct and efficient code.

6.e – Non coding exercises

1. What does `'Hi' * 3` produce?
2. What does `letters[1:3]` return for `letters = ['a','b','c','d']`?
3. What does `len([8, [], 'hello'])` return?
4. Which operator merges two dictionaries?
5. What does `r'\n'` represent?
6. Strings are mutable in Python. (True or false?)
7. `min()` and `max()` work only on numeric types. (True or false?)
8. Slicing includes the end index. (True or false?)
9. `+=` on a list modifies the list in place. (True or false?)
10. `id()` values are the same across all Python runs. (True or false?)
11. A raw string disables escape sequence processing.
12. `bytes` is a mutable sequence type. (True or false?)
13. `in` works on strings, lists, and tuples. (True or false?)
14. What is the result of `'Hello' + 'World'` ?
15. What is the result of `'ab' * 2`?
16. What is the result of `(1, 2) + (3,)`?
17. What is the result of `[1, 2] * 2` ?
18. What is the result of `len('Narendra')`?
19. What is the result of `'N' in 'Narendra'`?
20. What is the result of `[] in [8, [], 'hello']` ?
21. What is the result of `0 in [8, [], 'hello']`?
22. What is the result of `max((8, 6, 4.6, -9))`?
23. What is the result of `min(['apple', 'zebra', 'mango'])`?
24. What is the result of `{1, 2} | {3, 4}`?
25. What is the result of `{1: 'one'} | {2: 'two'}`?
26. What is the result of `letters = ['a','b','c','d']; letters[-1]`?
27. What is the result of `letters[0:4:2]`?
28. What is the difference between immutable and mutable types. Give one example of each.
29. Why does `id()` change after assigning a new value to a string variable but not after `append()` to a list?
30. What is the difference between `+` and `+=` on a list?
31. Why are raw strings useful? Give a practical example.
32. What is the difference between `str()`, `repr()`, and `print()`?

- 33. Explain slicing with a step. What does `::2` do?
- 34. Why does `'Hi ' * 3` work but `'Hi ' * 'x'` does not?
- 35. What is the difference between `in` for a string and `in` for a list?

8 – SETS & MAPPINGS

8.a – Learning Outcomes

By the end of this chapter, you will be able to:

- Define sets and dictionaries and explain their roles as unordered collections in Python.
- Explain the concept of hashability and why sets and dictionary keys require hashable objects.
- Differentiate between hashable and unhashable types (int, float, str, bool, tuple vs. list, dict, set).
- Create sets and dictionaries using appropriate syntax, including `set()` for empty sets and `{}` for empty dictionaries.
- Apply common set methods such as `add()`, `remove()`, `discard()`, `pop()`, and set operations like union, intersection, difference, and symmetric difference.
- Use set operators (`|`, `&`, `-`, `^`) as alternatives to their corresponding methods.
- Construct dictionaries and perform operations including accessing, adding, updating, and deleting key-value pairs.
- Utilize dictionary methods such as `keys()`, `values()`, and `items()` to iterate over dictionary contents.
- Identify the key differences between `lists`, `tuples`, `sets`, and `dictionaries` in terms of order, mutability, and uniqueness.
- Recognize common pitfalls when working with sets and dictionaries, such as attempting to add mutable objects or relying on element order.

8.b – Introduction

Python provides several built-in collection types, each designed for different purposes. While lists and tuples are **sequences** that maintain order and support indexing, sets and dictionaries are **mappings and sets** that use **hash tables** for efficient storage and lookup. Understanding these collections is essential for writing efficient Python programs that handle unique values and key-value relationships.

This chapter covers two important collection types:

- **Sets** – unordered collections of unique, hashable items
- **Dictionaries** – collections of key-value pairs where keys must be hashable

Both rely on the concept of hashability, which we will explore in detail.

8.c – Set

A set is an unordered collection of unique items.
A set may only contain hashable objects such as:

- `int`
- `float`
- `str`
- `bool`
- `tuples` (if all the items are hashable)
- `bytes`.

Interesting fact

You may use the `hash()` function to check if an object is hashable.

Sets cannot contain mutable type such as:

- `list`
- `dict`
- `set`
- `bytearray`

This restriction exists because set elements must be **hashable**, because Python uses hash value for fast lookups and to ensure uniqueness.

Notes

- Sets do **not** preserve order
- Duplicate values are **automatically removed**
- Set elements must be **hashable**
- Mutable objects like `list` or `dict` cannot be stored in a set.

8.c.1 – Creating sets

```
# empty set must use set() not {}
>>> a = set()
>>> a
set()

# set with a single item
>>> b = {0}
>>> b
{0}

# creating sets from other collections
```

```
>>> set([1, 2, 3])      #set from a list
{1, 2, 3}

>>> set((1, 2, 3))      #from a tuple with hashable items
{1, 2, 3}

>>> set({4: 'four', 'five': 5, 6: 'six'})  #from a dict (keys)
{4, 'five', 6}
```

8.c.1.1 – Hashability

A **hash value** is an integer used internally by Python to store objects efficiently in sets and dictionaries. Objects that are **hashable** can be used as set elements or dictionary keys.

```
>>> hash(3)
3
>>> hash('hello')
4800600899085869277
>>> hash([0,9])
Traceback (most recent call last):
  File "<python-input-10>", line 1, in <module>
    hash([0,9])
    ~~~~^~~~~~
TypeError: unhashable type: 'list'
```

List are mutable, so they cannot be hashed.

8.c.2 – Common Set Methods

`add(x)`

Add an element to this set

```
>>> s = {1, 2, 3}
>>> s.add(4)
>>> s
{1, 2, 3, 4}
```

```
clear()
```

Remove all elements from this set

```
>>> s = {1, 2, 3}
>>> s.clear()
>>> s
{}

```

```
copy()
```

Create and return a shallow copy of this set

```
>>> s = {1, 2, 'Alice'}
>>> t = s.copy()
>>> t
{1, 2, 'Alice'}

```

```
difference(x)
```

Return a new set with elements of this set except the ones in the given set arguments.

```
>>> {1, 2, 3}.difference({1, 2})
{3}

```

```
difference_update(iter)
```

Remove all elements from this set that are members of any of the given set arguments.

```
>>> s = {1, 2, 3}
>>> s.difference_update({1, 2})
{3}

```

`discard(x)`

Remove an element if it exists.

If the element does not exist, nothing happens.

```
>>> s = {'Alice', 'Bob', 'Cloe'}
>>> s.discard('Bob')
>>> s
{'Alice', 'Cloe'}
```

`intersection()`

Return a new set of elements that are members of this and the set argument(s).

```
>>> {1, 2, 3, 4}.intersection({3, 4, 5})
{3, 4}
```

`intersection_update()`

Updates the set so it contains only elements common to both sets.

```
>>> s = {1, 2, 3, 4}
>>> s.intersection_update({3, 4, 5})
>>> s
{3, 4}
```

`isdisjoint(x)`

Return True if their intersection is the empty set.

```
>>> {1, 2, 3, 4}.isdisjoint({'Alice', 'Bob'})
True
```

issubset()

Return True if all elements of this set are members of the specified set argument.

```
>>> t = {'Alice', 'Bob', 'Carl', 'Liz'}
>>> {'Alice', 'Bob'}.issubset(t)
True
```

issuperset()

Return True if all elements of the specified of the set argument are members of this set.

```
>>> {'Alice', 'Bob', 'Carl'}.issuperset({'Alice'})
True
```

pop()

Remove and return a random element from the set.

Raises `KeyError` if set is empty.

```
>>> s = {'Alice', 'Bob', 'Carl'}
>>> s.pop()
'Alice'
>>> s
{'Bob', 'Carl'}
```

remove(x)

Remove the specified element from the set.

Raises `KeyError` if the element is not present.

```
>>> s = {'Alice', 'Bob', 'Cloe'}
>>> s.remove('Bob')
>>> s
```

```
{'Alice', 'Cloe'}
```

`symmetric_difference()`

Return new set with elements in either this or the specified set argument, but not both.

```
>>> {1, 2, 3}.symmetric_difference({2, 3, 4})
{1, 4}
```

`symmetric_difference_update()`

Replace this set with the symmetric difference, i.e., elements in either this set or the specified set argument, but not both.

```
>>> s = {1, 2, 3}
>>> s.symmetric_difference_update({2, 3, 4})
>>> s
{1, 4}
```

`union()`

Return new set with all elements from both sets.

```
>>> {1, 2, 3, 4}.union({3, 4, 5})
{1, 2, 3, 4, 5}
```

`update()`

Update this set with all elements that are in this or any of the specified set arguments.

```
>>> s = {1, 2, 3, 4}
>>> s.update({3, 4, 5})
>>> s
{1, 2, 3, 4, 5}
```

A sequence is an ordered collection of items that supports operations such as indexing, slicing, iteration and membership testing. They form a core part of the python language and many built-in types are sequences.

Sequences may be categorized into two main groups: Immutable and mutable.

Set Operators

Python provides operators that correspond to common operations.

Operators	Method	Operator	Example
Union	<code>union()</code>		<code>A B</code>
Intersection	<code>intersection()</code>	&	<code>A & B</code>
Difference	<code>difference()</code>	-	<code>A - B</code>
Symmetric Difference	<code>symmetric_difference()</code>	^	<code>A ^ B</code>

8.d – Dictionary

A **dictionary** stores key-value pairs. Keys must be **hashable** (immutable), while values can be any type.

Information

The numeric types `int` and `float` are immutable as well as the set type `frozenset`.

```
>>> # Creating dictionaries
>>> empty_dict = {}
>>> grades = {'Alice': 85, 'Bob': 92, 'Charlie': 78}

>>> # Accessing values
>>> grades['Alice']
85

>>> # Adding/updating entries
>>> grades['David'] = 88
```

```
>>> grades['Alice'] = 90

>>> # Dictionary methods
>>> grades.keys()
dict_keys(['Alice', 'Bob', 'Charlie', 'David'])

>>> grades.values()
dict_values([90, 92, 78, 88])

>>> grades.items()
dict_items([('Alice', 90), ('Bob', 92), ('Charlie', 78), ('David', 88)])
```

8.e – Summary

This chapter introduced sets and dictionaries, two hash-based collections that provide efficient storage and lookup for unique values and key-value relationships.

Key takeaways:

- **Sets** are unordered collections of unique, hashable items. Duplicate values are automatically removed, and sets do not preserve order.
- **Hashability** is required for set elements and dictionary keys. Mutable types (`list`, `dict`, `set`, `bytearray`) cannot be hashed and therefore cannot be used as set elements or dictionary keys.
- **Tuples** are hashable **only if all their elements are hashable**. A tuple containing a list is not hashable.
- **Set operations** include union (`|`), intersection (`&`), difference (`-`), and symmetric difference (`^`), each with corresponding methods.
- **Sets provide O(1) membership testing**, making them ideal for checking whether an item exists in a collection.
- **Dictionaries** store key-value pairs and use hash tables for O(1) average-time lookups. Keys must be hashable; values can be any type.
- **Dictionary methods** such as `keys()`, `values()`, and `items()` provide views of the dictionary's contents for iteration.
- **Dictionaries preserve insertion order** (Python 3.7+), though this is an implementation guarantee rather than a fundamental property of the data structure.

- **Shallow copies** of sets can be made with `set.copy()` or `set(old_set)`. The `copy()` method is more readable .

8.f – Non coding exercises

1. What is the difference between a list and a tuple?
2. What is the difference between a set and a dictionary?
3. Which collection stores ordered and mutable items.
4. Which collection stores ordered and immutable items.
5. Which collection stores unique values only.
6. Which collection does not allow duplicate values?
7. Which collection stores key-value pairs.
8. Which collection stores key-value pairs?
9. Lists and tuples are identical because both store multiple values. (true or false?)
10. Dictionaries use numeric positions just like lists. (true or false?)
11. Every collection in Python allows duplicate values. (true or false?)
12. List the mutable types in python
13. List the immutable types in python
14. Why can't a list be used as a set element?
15. Why can't a dict be used as a dictionary key?
16. What does `hash()` return for a list? Why?
17. What is the result of `{1, 2, 3} | {3, 4, 5}`?
18. What is the result of `{1, 2, 3} & {3, 4, 5}`?
19. What is the result of `{1, 2, 3} - {3, 4, 5}`?
20. What is the result of `{1, 2, 3} ^ {3, 4, 5}`?
21. Why do sets not preserve order?
22. What is the time complexity of membership testing in a set? Why?
23. Why must dictionary keys be immutable?
24. What happens if you try to use a list as a dictionary key?
25. What is the difference between `remove()` and `discard()` for sets?
26. What is the difference between `pop()` for sets and `pop()` for lists?

9 – FILE I/O & ERRORS

9.a – Learning Outcomes

By the end of this chapter, you will be able to:

1. Explain why file I/O is essential for persisting data beyond a program's execution.
2. Open files using the `open()` function and describe the purpose of each parameter.
3. Differentiate between file modes ('r', 'w', 'a', 'x', 't', 'b') and select the appropriate mode for a given task.
4. Write text to a file using the `write()` method and ensure the file is properly closed.
5. Read file contents using `read()`, `readline()`, and `readlines()`, and explain the difference between each method.
6. Use the `with` statement to perform file I/O safely, explaining how it automatically manages resource cleanup.
7. Perform common file management operations using the `os` module, including checking file existence, deleting files, creating folders, and removing empty folders.
8. Identify and distinguish between common Python errors: `SyntaxError`, `NameError`, `TypeError`, `ValueError`, `IndexError`, `KeyError`, `FileNotFoundError`, and `ZeroDivisionError`.
9. Describe the purpose of `try`, `except`, `else`, and `finally` blocks in error handling.
10. Apply basic error handling techniques to write programs that respond gracefully to runtime errors.
11. Recognize the risks of using the `os` module for direct filesystem manipulation.

9.b – Introduction

Writing to a file is the primary way to **persist data** after a program terminates. Without file input/output (I/O), all data created by a program is lost when the program ends. In this course, we will work **only with text files** — everything written to or read from a file will be strings.

This chapter covers two essential topics:

- **File I/O** – reading from and writing to files on disk
- **Errors** – understanding what can go wrong and how to handle it

Understanding both topics is critical for building robust programs that can save their results and recover gracefully from unexpected situations.

Important: When working with files, you must always close the file after completing all input or output operations. Failure to do so can result in data loss or corruption. The `with` statement (covered in section 9.a.4) is the preferred way to ensure files are closed automatically.

9.c – Working with files

Writing to file is the primary way to persist data after a program finish running. In this course we will work ONLY be using text files i.e. everything written or read from a file will be strings.

9.c.1 – Opening a file

The `open()` function opens a file and return a **stream** (file handle in other languages). This stream can be used to write to or read from a file. You must always close the stream after completing all input or output operations.

```
f = open(
    «filename»,          #pathname to the file
    [mode = 'r'],        #read/write mode
    [encoding = None],    #ignore this argument
    [newline = None]     #also ignore this argument
)
```

The file should always be closed after you finish using it.

9.c.2 – Writing to a file

The `write()` method sends string to the opened file.

```
f = open('test.txt', 'wt')    # write text mode
f.write('file created by Narendra')
f.close()                    #closes the file
```

9.c.3 – Reading from a file

Three commonly used methods:

Method	Description
<code>read()</code>	returns the entire file content as a single string.
<code>readline()</code>	returns the current line in the file as a single string and move the pointer to the next line. A subsequent read will get the next line.
<code>readlines()</code>	returns the entire file content as string items of a list.

```
f = open('test.txt')      #no mode is specified
first = f.readline()      #reads the first line
rest = f.read()           #read the rest of the file
f.close()                 #closes the file
```

9.c.4 – A more elegant way of reading/writing a file

The `with` statement is a more modern way to performing file IO.

```
with open('test.txt') as f:
    for line in f:
        print(line)      #automatically the file is closed and all
                          #associated resources released
```

The file closes automatically once processing move out of the `with` block.

9.c.5 – File Modes

Mode	Meaning
<code>'r'</code>	Read (default)
<code>'w'</code>	Write (overwrites file)
<code>'a'</code>	Append to file
<code>'x'</code>	Creates a new file
<code>'t'</code>	Text mode (default)

'b'	Binary mode
-----	-------------

9.d – Operation System File Management

You will need to import the `os` module to access the file management capabilities of Python. Fortunately, this module does not need to be downloaded, because it is a part of the standard Python installation.

Be very careful when using this module, because you will have direct access to your computer filesystem and you have the ability to corrupt your machine.

9.d.1 – Checking for the existence of a file

You will use the `exist()` method to check if the path (either file or folder) is present.

```
>>> import os
>>> os.path.exists('person.dat')
True
>>> os.path.exists('foo.dat')
False
```

9.d.2 – Deleting a file

You will use the `remove()` method to delete a file.

```
>>> if os.path.exists('person.dat'):
    os.remove('person.dat')
```

9.d.3 – Deleting a folder

You will use the `rmdir()` method to delete a folder. Please be aware that you are only able to remove empty folders.

```
os.rmdir('temp')
```

9.d.4 – Creating a folder

You will use the `mkdir()` method to create a folder.

```
os.mkdir('foo')
```

9.e – Errors

Syntax errors happen when your code does not make any sense to the computer. In other words, it encounters something that it does not expect! This can happen because you've misspelled something or there is one too many brackets or a missing comma or even the incorrect indentation level

9.e.1 – Syntax Error

Occurs when Python encounters invalid syntax.

```
print(12 + 4))
File "<python-input-0>", line 1
    print(12 + 4))
                ^
SyntaxError: unmatched ')'
```

9.e.1 – Name Error

This happens when there is a variable with a name that the computer does not recognize. It's usually because you've misspelled the name of a variable you created earlier.

```
my_number = 4
my_Number + 2
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    my_Number + 2
    ^^^^^^^^^^^
NameError: name 'my_Number' is not defined. Did you mean: 'my_number'?
```

Remember variable names are case sensitive!

9.e.2 – Type Error

This happens when an operation is performed on an inappropriate type.

```
>> 'H' + 5
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    'H' + 5
    ~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

9.e.3 – Value Error

This happens when a function receives an argument of correct type but inappropriate value.

```
>>> import math
>>> math.sqrt(-1)
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    math.sqrt(-1)
    ~~~~~~^~~~~
ValueError: math domain error
```

9.e.4 – Index Error

This happens when a sequence subscript is out of range.

```
>>> a = [1, 2, 3, 4, 5]
>>> a[5]
Traceback (most recent call last):
  File "<python-input-4>", line 1, in <module>
    a[5]
    ~^^^
IndexError: list index out of range
```

9.e.5 – Key Error

This happens when a dictionary key is not found.

```
>>> a = {1: 'one', 2: 'two', 3: 'three'}
```

```
>>> a[0]
Traceback (most recent call last):
  File "<python-input-16>", line 1, in <module>
    a[0]
    ~^^^
KeyError: 0
```

9.e.6 – FileNotFoundError

This happens when a file or directory is requested but doesn't exist.

```
>>> f = open('missing.txt', 'rt')
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    f = open('missing.txt', 'rt')
FileNotFoundError: [Errno 2] No such file or directory: 'missing.txt'
```

9.e.7 – Zero Division Error

This happens when you try to divide by zero. This is something that is mathematically impossible so Python will also complain.

```
5 % 0
Traceback (most recent call
last): File '<stdin>', line 1,
ZeroDivisionError: integer division or modulo by zero
```

9.e.8 – Error handling

Error handling is an advanced topic and might not be a part of the course.

```
try:
    python statement that can cause an error situation

except «type_of_error1»:
    do something

except «type_of_error2»:
    do something
```

9.e.9 – A Practical Example

```
try:
    # Code that might cause an error
    number = int(input("Enter a number: "))
    result = 10 / number
    print(f"Result: {result}")

except ValueError:
    # Handle invalid number conversion
    print("That's not a valid number!")

except ZeroDivisionError:
    # Handle division by zero
    print("Cannot divide by zero!")

except Exception as e:
    # Handle any other exception
    print(f"An error occurred: {e}")

else:
    # Runs if no exception occurred
    print("Operation successful!")

finally:
    # Always runs (cleanup code)
    print("Done!")
```

9.f – Summary

This chapter covered two essential topics for writing robust Python programs: file I/O and error handling.

9.f.1 – File I/O:

- `open()` creates a file stream. Always close the file when done.

- **File modes** control how the file is opened: `'r'` (read), `'w'` (write/overwrite), `'a'` (append), `'x'` (create new), `'t'` (text), `'b'` (binary).
- `write()` writes a string to a file.
- `read()`, `readline()`, and `readlines()` read file contents in different ways.
- **The `with` statement** is the preferred way to handle file I/O because it automatically closes the file, even if an error occurs.
- **The `os` module** provides file management operations: `exists()`, `remove()`, `rmdir()`, `mkdir()`. Use with caution.

9.f.2 – Errors:

- `SyntaxError` – invalid syntax (e.g., unmatched parentheses)
- `NameError` – undefined variable (often a misspelling or case error)
- `TypeError` – operation on an inappropriate type
- `ValueError` – correct type but inappropriate value
- `IndexError` – sequence index out of range
- `KeyError` – dictionary key not found
- `FileNotFoundError` – file or directory does not exist
- `ZeroDivisionError` – division by zero

9.f.3 – Error Handling:

- `try/except` blocks allow you to catch and handle exceptions gracefully.
- `else` runs if no exception occurred.
- `finally` always runs, making it ideal for cleanup code.
- Catch specific exceptions before general ones.

9.g – Non-Coding Exercises

1. What is the primary purpose of file I/O in a program?
2. What does the `open()` function return?
3. Why must you always close a file after using it?
4. What is the default mode for `open()`?
5. What is the difference between `'w'` and `'a'` file modes?
6. What does the `'x'` file mode do?
7. What is the difference between `read()`, `readline()`, and `readlines()`?
8. What happens if you call `readline()` after `read()`?
9. Why is the `with` statement preferred over manually calling `close()`?
10. What module must you import to perform file management operations?

11. What method checks if a file or folder exists?
12. What method deletes a file?
13. What method creates a folder?
14. What method deletes a folder? What restriction applies?
15. What is the difference between a syntax error and a runtime error?
16. What error occurs when you misspell a variable name?
17. What error occurs when you add a string and an integer?
18. What error occurs when you access an index that doesn't exist?
19. What error occurs when you access a dictionary key that doesn't exist?
20. What error occurs when you try to open a file that doesn't exist?
21. What error occurs when you divide by zero?
22. What is the purpose of the `try` block?
23. What is the purpose of the `except` block?
24. What is the purpose of the `else` block in a `try/except` structure?
25. What is the purpose of the `finally` block?
26. Why is it better to catch specific exceptions rather than a general `Exception`?
27. What happens if an exception occurs inside a `with` block? Is the file still closed?
28. Why should you be careful when using the `os` module?

10 – PACKAGES/MODULES

10.a – Learning Outcomes

By the end of this chapter, you will be able to:

- **Define** what a module is and explain its role in organizing and reusing Python code.
- **Differentiate** between built-in modules, external modules, and user-created modules.
- **Create** a simple Python module containing variables, functions, and classes.
- **Import** a module using the `import` keyword and access its contents using dot notation.
- **Import** specific items from a module using the `from ... import ...` syntax.
- **Use** the wildcard (*) import and explain why it is generally discouraged.
- **Alias** modules and imported items using the `as` keyword.
- **Remove** a module from the current namespace using the `del()` function.
- **Define** what a package is and explain the role of the `__init__.py` file.
- **Recognize** the difference between the standard library, PyPI packages, and local modules.

10.b – Introduction

As programs grow larger, placing all code in a single file becomes impractical. Python solves this problem through **modules** and **packages**, which allow you to organize code into reusable, maintainable units.

A **module** is a single Python file containing definitions and statements. A **package** is a directory containing multiple modules and a special `__init__.py` file. Together, they form the foundation of Python's code organization and reuse strategy.

This chapter covers:

- **Modules** – creating, importing, and using modules
- **Packages** – understanding how Python organizes modules into directories

Note: In this course, we constrain ourselves to modules available in the standard Python installation. We will avoid downloading and installing external packages.

10.c – Modules

A module is a file containing Python code that can define:

- variables
- functions
- classes

Modules allow you to organize and reuse/share code across different programs or projects.

10.c.1 – Examples of built-in modules

These are pre-installed during a standard python installation:

- `os`
- `abc`
- `random`
- `math`
- `json`
- `datetime`

10.c.2 – External modules

These need to be installed from pypi.org using `pip`:

- `matplotlib`
- `numpy`
- `pandas`
- `tensorflow`
- `flask`

10.c.3 – Creating your own Module

We will create a python code file that has a variable declaration and two function definitions. If the file is in the same folder that you are running the Python interpreter, then your usage does not need the path of the name, just use the name.

```
'''
This module serves to illustrate the purpose
and use of modules in the python language.
'''
pi = 3.141_592

def greet(name):
    return f'Hello {name}'
```

```
def calculate_area_of_triangle(base, height):  
    return base * height * 0.5
```

10.c.4 – Importing/Loading a Module

In order to access the code in a module, you will have to load that module. We will use the python `import` keyword with the name of the file to import.

```
>>> dir()  
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',  
 '__package__', '__spec__']  
  
>>> import module01  
>>> dir()  
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',  
 '__package__', '__spec__', 'module01']  
  
>>> dir(module01)  
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__',  
 '__name__', '__package__', '__spec__', 'calculate_area_of_triangle',  
 'greet', 'pi']
```

Note: The `dir()` function without arguments lists names in the current namespace. With a module argument, it lists the names defined in that module.

10.c.4.1 – Importing specific items from a module

You can import a specific thing from a module. e.g. a function/class/constant You do this with the `from` keyword. It can save you from having to type the same thing many times.

```
>>> from module01 import pi, greet  
>>> pi  
3.141592  
  
>>> greet('Hao')  
'Hello Hao'
```

```
>>> calculate_area_of_triangle(4, 7)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'calculate_area_of_triangle' is not defined
```

Note: When you use `from module01 import pi, greet`, only `pi` and `greet` are brought into the current namespace. `calculate_area_of_triangle` is **not** available unless explicitly imported.

10.c.4.2 – Importing Everything

You can use the wildcard (*) to import everything from a module. Beware, this usually reduces code readability and also over-crowd the current workspace.

```
>>> from module01 import *
>>> pi
3.141592

>>> greet('Hao')
'Hello Hao'

>>> calculate_area_of_triangle(4, 7)
14.0
```

Best Practice: Avoid wildcard imports in production code. They make it unclear where names come from and can accidentally overwrite existing names.

10.c.4.3 – Aliasing

You can use the `as` keyword to give your module a different name. Then you will use this alias to access the module. This is useful if the name of the module changes and you have existing code that references the old module. You will use an alias also if it is tedious to type the full name of the module.

```
>>> from module01 import calculate_area_of_triangle as area
>>> area(4, 7)
```

14.0

```
>>> calculate_area_of_triangle(4, 7)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'calculate_area_of_triangle' is not defined
```

Note: Once aliased, the original name is no longer available in the current namespace. You must use the alias.

10.c.5 – Using the Module

Statement to use the module will depend on how the module was imported.

```
>>> import module01
>>> module01.pi
3.141592
>>> module01.greet('Hao')
'Hello Hao'
```

Note: When you import the module itself (not specific items), you must prefix every access with the module name followed by a dot (.).

10.c.6 – Removing a Module

Use the `del()` function to remove a module from the current namespace.

```
del(module01)
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__']
```

Note: `del()` removes the **name binding** from the current namespace. It does **not** delete the module file from disk. The module can be re-imported later.

10.d – Packages

A **package** is a directory that contains multiple Python files (modules) and a special file named `__init__.py`. The `__init__.py` file indicates to Python that the directory should be treated as a package.

In this course we constrain ourselves to the modules that are available in the standard python installation, so we will avoid downloading and installing extra packages.

10.d.1 – Package Structure

```
my_package/  
  __init__.py  
  module_a.py  
  module_b.py  
  subpackage/  
    __init__.py  
    module_c.py
```

- `__init__.py` can be empty, or it can contain initialization code for the package.
- Modules within a package are imported using dot notation: `import my_package.module_a`

10.d.2 – Creating your own Package

This is not in the scope of this course. However, understanding the structure helps you navigate existing packages.

10.e – Summary

This chapter introduced modules and packages, the fundamental building blocks of code organization and reuse in Python.

Key takeaways:

- **Modules** are Python files (`.py`) containing variables, functions, and classes that can be imported and reused across programs.
- **Built-in modules** (`os`, `random`, `math`, `json`) come pre-installed with Python.
- **External modules** (`numpy`, `pandas`, `flask`) must be installed from PyPI using `pip`.
- `import module_name` loads the entire module; access items with `module_name.item`.
- `from module_name import item1, item2` loads specific items into the current namespace.

- `from module_name import *` loads everything — convenient but discouraged due to readability and namespace pollution.
- `import module_name as alias` or `from module_name import item as alias` creates a shorter or alternative name.
- `del(module_name)` removes the module name binding from the current namespace (does not delete the file).
- **Packages** are directories containing multiple modules and an `__init__.py` file, which tells Python to treat the directory as a package.
- **Packages enable hierarchical organization** of related modules, making large codebases manageable.
- **Best practice:** Prefer explicit imports (`import module` or `from module import specific_item`) over wildcard imports for clarity and maintainability.

10.f – Non coding exercises

1. What is a module?
2. What three things can a module define?
3. Why are modules useful in programming?
4. Name three built-in Python modules.
5. Name three external Python modules and where they are installed from.
6. How do you create your own module?
7. What keyword is used to import a module?
8. What is the difference between `import module01` and `from module01 import pi`?
9. What happens if you try to use a function that was not imported when using `from module01 import pi, greet`?
10. What does the wildcard (*) do in an import statement?
11. Why is wildcard importing discouraged?
12. What keyword is used to alias a module or imported item?
13. How do you access a variable from a module that was imported with `import module01`?
14. How do you access a variable from a module that was imported with `from module01 import pi`?
15. What does `del(module01)` do? Does it delete the file from disk?
16. What is a package?
17. What special file must a directory contain to be treated as a package?
18. What is the purpose of `__init__.py`?
19. How do you import a module from within a package?

20. What is the difference between the standard library and external packages?
21. Why does this course avoid external packages?
22. What does `dir()` return when called with no arguments?
23. What does `dir(module01)` return?
24. True or false: A module can contain classes.
25. True or false: A package can contain subpackages.
26. True or false: `del(module01)` permanently deletes the module from your computer.
27. True or false: `from module01 import *` is the recommended way to import modules.
28. True or false: An `__init__.py` file must contain code to be valid.
29. Why might you use an alias for a module?
30. What is the difference between a module and a package?

11 – OBJECT ORIENTED PROGRAMMING

11.a – Learning Outcomes

By the end of this chapter, you will be able to:

- **Define** what a class is and explain its role as a blueprint for creating objects.
- **Create** a Python class using the `class` keyword and follow PascalCase naming conventions.
- **Instantiate** objects from a class and explain how each object maintains its own state.
- **Implement** the `__init__()` method to initialize instance variables upon object creation.
- **Differentiate** between instance variables, class variables, instance methods, class methods, and static methods.
- **Apply** the `@classmethod` and `@staticmethod` decorators appropriately and explain when to use each.
- **Explain** the four pillars of Object-Oriented Programming: encapsulation, inheritance, polymorphism, and abstraction.
- **Implement** encapsulation using private variables and getter/setter methods.
- **Construct** class hierarchies using inheritance and override parent methods in child classes.
- **Demonstrate** polymorphism by defining methods with the same name across different classes.
- **Use** abstract base classes from the `abc` module to enforce consistent design.
- **Evaluate** when to use OOP principles to make programs more modular, reusable, and maintainable.

11.b – Introduction

Object-Oriented Programming (OOP) is a programming paradigm that organizes code around **objects** rather than functions and logic. An object bundles together **data** (attributes) and **behavior** (methods) into a single unit.

OOP is built around four fundamental principles — often called the **four pillars**:

1. **Encapsulation** – bundling data and methods, controlling access
2. **Inheritance** – creating new classes from existing ones
3. **Polymorphism** – same method name, different behavior
4. **Abstraction** – hiding complexity, exposing essentials

These principles help developers build programs that are **modular**, **reusable**, and **easier to maintain**.

Why OOP matters: As programs grow, managing complexity becomes the central challenge. OOP provides a structured way to model real-world entities, reduce duplication, and isolate changes so they don't ripple through the entire codebase.

11.c – Classes and Objects

A **class** can be considered as a software **blueprint** that defines the **properties** (attributes or states) and **behavior** (methods or actions) of an entity.

It serves as a template for creating an **object**, which is an individual **instance** of that class.

Each object created from a class maintains its **own state**, even though all share the same structure defined by the class.

11.c.1 – Defining a Python Class

You create a class using the `class` keyword. Note, class names in Python usually follows **PascalCase** naming style.

The following creates a minimalistic class

```
class Account:  
    pass
```

Note: The `pass` statement is a placeholder that allows the class body to be empty without causing a syntax error.

11.c.2 – Instantiating the Class

Create an object by calling the class like a function.

```
>>> obj = Account()  
>>> type(obj)  
<class '__main__.Account'>
```

Interesting fact

Newer language uses the `new` operator to instantiate a class.

11.c.3 – The `__init__` method

The `__init__()` method is called every time a new object is created from the class. It is referred to as the constructor in other languages.

Since the constructor is called immediately upon object creating, it is the best place to put initialization code or logic to make the object usable as soon as possible.

Interesting fact

The `__init__()` method is the constructor of the class, however it does not create an object, the runtime does that. The `__init__()` is invoked immediately after object creation.

```
class Account:
    def __init__(self, name, number, balance = 0):
        self.holder = name
        self.number = number
        self.balance = balance

>>> acct = Account('Arben Tapia', '1234', 100)
>>> acct.holder
'Arben Tapia'
>>> acct = Account('Ilia Nika', '1235', 200)
>>> acct.holder
'Ilia Nika'
```

11.c.4 – Instance Variables

Instance variable are values that are **unique to each object**. These are what makes each object different from other objects. Variable created with any instance method that are attached to the `'self'` pronoun are considered as instance variables.

Instance variables are most often declared in the `__init__()__()` because this method is called immediately upon object creation, hence any subsequent methods invoked will have access to those variables.

```
class Car:
    def __init__(self, range):
        self.range = range
```

```
def drive(self):
    return f'I can drive {self.range}km'

>>> honda = Car(450)
>>> honda.drive()
'I can drive 450km'
```

11.c.5 – Instance Methods

Instance methods are functions declared inside the class that operate on instance data. They always take `self` as their first parameter, which refer to the current object.

```
class Car:
    def drive(self):
        return 'I can drive 500km'

>>> honda = Car()
>>> honda.drive()
I can drive 500km
```

11.c.6 – Class Variables

Class variables are shared by **all instances** of a class. Changing a class variable affects all objects.

```
class Person:
    species = 'Homo sapiens'      # Class variable

    def __init__(self, name):
        self.name = name         # Instance variable

>>> ilia = Person('Ilia')
>>> ana = Person('Ana')
>>> ilia.species
'Homo sapiens'
```

```
>>> ana.species
'Homo sapiens'

# Change affects all instances
>>> Person.species = 'Human'
>>> ilia.species
'Human'
```

Important: If you assign `ilia.species = 'Something'`, you create a **new instance variable** that shadows the class variable for that one object. To change the class variable for all objects, assign to `Person.species`.

11.c.7 – Class Methods

These are functions that belongs to a class rather than individual objects. Class methods operate on the class itself rather than instances. They're decorated with `@classmethod` and take `cls` as the first parameter.

Class methods are commonly used to modify class variables or **create alternative constructors**.

```
class Person:
    species = 'Homo sapiens'

    def __init__(self, name):
        self.name = name

    @classmethod
    def change_species(cls, new_species):
        cls.species = new_species

    @classmethod
    def create_anonymous(cls):
        return cls('Anonymous')    # Factory method

>>> Person.change_species('Human')
>>> Person.species
'Human'
```

```
>>> anonymous = Person.create_anonymous()
>>> anonymous.name
'Anonymous'
```

11.c.8 – Static Methods

Static methods are utility functions within a class that **don't access instance or class data**. They're decorated with `@staticmethod`.

```
class MathOperations:
    @staticmethod
    def add(x, y):
        return x + y

    @staticmethod
    def multiply(x, y):
        return x * y

>>> MathOperations.add(5, 3)
8
>>> MathOperations.multiply(4, 2)
8
```

11.c.9 – Key Concepts

Concept	Description	Example
Class	Blueprint for creating objects	<code>class Dog:</code>
Object	Instance of a class	<code>my_dog = Dog()</code>
Instance Variable	Unique to each object	<code>self.name = name</code>

Class Variable	Shared by all instances	<code>species = 'Canine'</code>
Instance Method	Operates on instance data	<code>def bark(self):</code>
Class Method	Operates on class data	<code>@classmethod</code>
Static Method	Utility function	<code>@staticmethod</code>
Inheritance	Child class inherits from parent	<code>class Puppy(Dog):</code>

11.c.10 – Comparison: Instance vs. Class vs. Static Methods

Feature	Instance Method	Class Method	Static Method
First parameter	<code>self</code>	<code>cls</code>	None
Decorator	None	<code>@classmethod</code>	<code>@staticmethod</code>
Accesses instance data?	Yes	No	No
Accesses class data?	Yes (via <code>self</code>)	Yes (via <code>cls</code>)	No
Call on	Instance	Class or instance	Class or instance
Common use	Behavior on object state	Factory methods, class state	Utility function

11.d – The Four Pillars of Object-Oriented Programming

Object-Oriented Programming (OOP) is built around four fundamental principles often referred to as the four pillars:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

These principles help developers build programs that are **modular**, **reusable**, and **easier to maintain**.

11.b.1 – Encapsulation

Encapsulation is the practice of **bundling data (variables) and the methods that operate on that data** inside a class. It also involves **restricting direct access** to certain parts of an object.

In Python, encapsulation is commonly implemented using **private** or **protected** variables and **getter/setter** methods.

```
class BankAccount:

    def __init__(self, balance):
        self.__balance = balance    # private variable

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

Usage:

```
>>> acct = BankAccount(100)
>>> acct.deposit(50)
>>> acct.get_balance()
150
```

The variable `__balance` is intended to be accessed only through the class methods.

11.b.1.1 – Benefits of encapsulation:

1. Protects internal object data
2. Prevents accidental modification
3. Makes code easier to maintain

11.b.1.1 – Python naming conventions:

- `_variable` (single underscore) – "protected" by convention; a hint to other developers
- `__variable` (double underscore) – name-mangled to `__ClassName__variable`; not truly private but harder to access accidentally
- `__variable__` (double underscore on both sides) – reserved for Python's special methods (dunder methods)

11.b.2 – Inheritance

Inheritance allows a class to inherit attributes and methods from another class.

This promotes code reuse and allows developers to build hierarchies of related classes.

The class being inherited from is called the **parent class** (or superclass).

The class that inherits is called the **child class** (or subclass).

```
class Animal:
    def speak(self):
        print("Some sound")

class Dog(Animal):
    def speak(self):
        print("Bark")
```

Usage:

```
>>> d = Dog()
>>> d.speak()
Bark
```

Here, `Dog` inherits from `Animal` and **overrides** the `speak()` method.

11.b.2.1 – Benefits of inheritance:

1. Code reuse
2. Logical class hierarchies
3. Easier program organization

Note: Python supports **multiple inheritance** (a class can inherit from more than one parent), unlike some languages. Use it carefully to avoid the "diamond problem."

11.b.3 – Polymorphism

Polymorphism means “many forms.” It allows different classes to implement methods with the **same name but different behavior**.

```
class Dog:
    def speak(self):
        return "Bark"

class Cat:
    def speak(self):
        return "Meow"
```

Usage:

```
animals = [Dog(), Cat()]

for animal in animals:
    print(animal.speak())
```

Output:

```
Bark
Meow
```

Even though both classes define a method named `speak()`, each object behaves differently.

11.b.3.1 – Benefits of polymorphism:

1. Flexible code
2. Easier to extend programs
3. Supports interchangeable objects

Note: Polymorphism in Python is often achieved through **duck typing** — "If it walks like a duck and quacks like a duck, it's a duck." Python doesn't require a common parent class for polymorphism to work; it only requires that the object has the method being called.

11.b.4 – Abstraction

Abstraction means hiding complex implementation details and exposing only the essential features of an object.

Python often implements abstraction using **abstract base classes** from the `abc` module.

Example:

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass
```

Child classes must implement the abstract method:

```
class Rectangle(Shape):

    def __init__(self, width, height):
        self.width = width
```

```
self.height = height

def area(self):
    return self.width * self.height
```

Usage:

```
>>> r = Rectangle(5, 3)
>>> r.area()
15
```

The Shape class defines what must be implemented, but not how it should be implemented.

11.b.4.1 – Benefits of abstraction:

1. Simplifies modeling, designing, implementing and testing complex systems
2. Forces consistent design
3. Improves maintainability

Important: You cannot instantiate an abstract class directly. Attempting to do so raises a `TypeError`. Abstract classes exist only to be inherited from.

Summary of the Four Pillars

Pillar	Description	Key Idea
Encapsulation	Bundling data and methods together and controlling access	Protect internal state
Inheritance	Creating new classes from existing ones	Reuse and extend
Polymorphism	Same method name behaving differently across classes	One interface many form
Abstraction	Hiding complexity and exposing essential features	Define what, not how

11.e – Summary

This chapter introduced Object-Oriented Programming (OOP), a paradigm that organizes code around objects that bundle data and behavior.

Key takeaways:

- **Classes** are blueprints that define attributes (data) and methods (behavior). **Objects** are individual instances of a class.
- `__init__()` is the constructor — called automatically when an object is created — and is the best place to initialize instance variables.
- **Instance variables** are unique to each object; class variables are shared by all instances.
- **Instance methods** take `self` and operate on instance data. Class methods take `cls`, are decorated with `@classmethod`, and operate on class data. Static methods take no special first parameter, are decorated with `@staticmethod`, and are utility functions.
- **Encapsulation** bundles data and methods and restricts direct access using private variables and getter/setter methods.
- **Inheritance** allows a child class to inherit attributes and methods from a parent class, promoting code reuse.
- **Polymorphism** allows different classes to implement methods with the same name but different behavior.
- **Abstraction** hides complex implementation details and exposes only essential features, often using abstract base classes from the `abc` module.
- **The four pillars** — encapsulation, inheritance, polymorphism, and abstraction — work together to make programs modular, reusable, and maintainable.

11.e.1 – Quick Reference: Defining a Class

```
class ClassName:
    class_variable = value          # Shared by all instances

    def __init__(self, param):      # Constructor
        self.instance_var = param  # Instance variable
```

```
def instance_method(self):      # Operates on instance
    return self.instance_var

@classmethod
def class_method(cls):          # Operates on class
    return cls.class_variable

@staticmethod
def static_method():            # Utility function
    return "No access to instance or class"
```

11.f – Non coding exercises

1. What is a class?
2. What is an object?
3. What is the difference between a class and an object?
4. What naming convention is used for class names in Python?
5. What is the purpose of the `__init__()` method?
6. What is another name for the `__init__()` method in other languages?
7. What is an instance variable?
8. What is a class variable?
9. What is the difference between an instance variable and a class variable?
10. What is an instance method?
11. What is the first parameter of an instance method?
12. What is a class method?
13. What decorator is used for a class method?
14. What is the first parameter of a class method?
15. What is a static method?
16. What decorator is used for a static method?
17. What is the difference between a class method and a static method?
18. When would you use a class method instead of an instance method?
19. When would you use a static method?
20. What are the four pillars of OOP?
21. What is encapsulation?
22. How is encapsulation implemented in Python?
23. What is the difference between `_variable` and `__variable` in Python?
24. What is inheritance?

25. What is the parent class called? What is the child class called?
26. What does it mean to "override" a method?
27. What is polymorphism?
28. How does Python's duck typing relate to polymorphism?
29. What is abstraction?
30. What module provides abstract base classes in Python?
31. What decorator is used to define an abstract method?
32. Can you instantiate an abstract class directly? Why or why not?
33. True or false: A class can have both instance variables and class variables.
34. True or false: Changing a class variable affects all instances.
35. True or false: A static method can access instance variables.
36. True or false: A class method can access class variables.
37. True or false: Inheritance promotes code reuse.
38. True or false: Polymorphism requires a common parent class in Python.
39. True or false: Encapsulation protects internal object data.
40. True or false: Abstraction hides complexity and exposes essential features.

Appendix

Docstring

APPENDICES

a – Programming Paradigms

This is the fundamental style or approach to writing code. It guides the design and implementation of software. You can think of it as a guiding philosophy or blueprint for building software. Each paradigm emphasizes certain concepts and techniques, and they significantly influence the structure and behavior of the code.

There is no modern programming language that support any paradigm fully (i.e., 100%), usually one language will support more than one paradigm to varying degree.

a.1 – Imperative Programming

This is one of the oldest and most straightforward paradigms. It emphasizes how to change the program's state (values/variables) step-by-step using commands.

Useful fact

Some practitioners lump Imperative and Procedural into a single paradigm.

Key Concepts:

- Variables: Store data.
- Assignment: Changing the value of a variable.
- Control Flow: Structures like loops and conditional statements (if/else) that determine the order of execution.

Examples

C, C++, Pascal

a.2 – Procedural Programming

This is an extension of imperative programming. It organizes code into reusable procedures or functions thus more manageable code base.

Key Concepts:

- Functions: Blocks of code that perform specific tasks and can be called multiple times.
- Modularity: Breaking down a program into smaller, more manageable units.

Examples

C, Pascal, FORTRAN

a.3 – Object-Oriented Programming (OOP)

This models real-world entities as objects that encapsulates both data and behavior. (properties / attributes) and methods (behaviors). Sometimes both properties and methods are referred to as attributes.

Key Concepts:

- Classes: Blueprints for creating objects.
- Objects: Instances of classes.
- Encapsulation: Bundling data (attributes) and methods that operate on that data within a single unit (object).
- Inheritance: Creating new classes (child classes) from existing ones (parent classes), inheriting their properties and methods.
- Polymorphism: The ability of objects to take on many forms.

Examples

Java, Python, C++, C#

a.4 – Functional Programming

This paradigm treats computation as the evaluation of mathematical functions. Emphasizes immutability (unchanging data) and avoids side effects.

Key Concepts:

- Pure Functions: Always produce the same output for the same input and have no side effects.
- Higher-Order Functions: Functions that take other functions as arguments or return functions as results.
- Recursion: Defining a function in terms of itself.
- Data Immutability:

Examples

Haskell, Lisp, Scala, Scheme, Erlang

a.5 – Declarative Programming

Here you specify the desired outcome, not the steps to achieve it.

Key Concepts:

- Logic Programming: Represents knowledge as facts and rules. (Example: Prolog)

- Dataflow Programming: Describes how data flows through a network of processes.

Examples

SQL, HTML, CSS

Why are Programming Paradigms Important?

Problem-Solving: Different paradigms offer unique ways to approach and solve problems.

Code Readability and Maintainability: Choosing the right paradigm can make your code more understandable and easier to maintain.

Learning New Languages: Understanding paradigms makes it easier to learn new programming languages.

Team Collaboration: Knowing your team's paradigms helps with communication and collaboration.

a.6 – The Best Paradigm

Personally, I don't feel that there is a best programming paradigm, your philosophy will be driven by the problem that needs to be solved, the available resources, such as existing expertise, the library and language support and if there is a partial implementation already in place.

b – What can be done with Python?

b.1 – Machine Learning and Artificial Intelligence

Libraries:

TensorFlow, PyTorch, scikit-learn, Keras

Use Cases:

1. Building and training machine learning models.
2. Natural language processing (NLP).
3. Computer vision.

4. Reinforcement learning.
5. Deploying and scaling machine learning models.

c – Why use Python?

d – Why you might consider another language?

Coding in another language might be preferred for the following reasons:

Performance Optimization: Certain tasks such as complex mathematical computations, low-level operations or real-time processing might be better done in C, C++ or Rust

Access to Specialized Libraries and Tools:

Hardware and System-Level Access: For tasks that require direct hardware or low-level system operations such as device drivers, accessing hardware sensors or performing low-level network operations can be more efficiently done in C++.

Resource Constraints: In resource-constrained environment, such as embedded systems or mobile devices using a more efficient language can be crucial.

Parallel and Concurrent Processing: For tasks that require high levels of concurrency or parallelism, languages like C++, Go or Rust might be more suitable due to their better support for these paradigms.

Security and Safety:

Memory Management: Languages like C++ and Rust provide fine-grained control over memory which is critical for performance and security.

DOCSTRING

Docstrings are similar in function to comments but are more formalized. It acts as documentation for functions, methods, classes, modules and packages. Docstrings are delimited with triple quotes whilst comments start with a # and goes to the end of the line.

Python supports the following data types:

3.a – None

None is a special constant, that is un-equal to everything except to another None. This is similar to the Null object in C#.

Fun fact

C# has 13 primitive numeric types

```
>>> None == 0
False
>>> None == []
False
>>> None == False
False
>>> x = None
>>> y = None
>>> x == y
True
```

It is an object that has its own datatype.

```
>>> type(None)
<class 'NoneType'>
```

DEBUGGING

If your program does not run, or crashes when it runs, or does not work as expected then your code has bugs (errors) in it. The process of identifying and fixing these bugs is called debugging. Debugging is an essential part of the software development process; it can be tricky but there are tools (provided by python and some provided by the IDE) that can make this process less painful.

Types of Errors

Errors occurs when the code does not behave as the programmer expects. There are three main types of errors:

Syntax Errors:

These errors occur when you write code that does not follow the language grammar rules such as missing parenthesis, misplaced colon or improper pairing of quotations. This kinds of errors as the easiest to find (and possible fix) because during development your IDE will flag most of them with squiggly colored lines. At runtime, the python interpreter locates them for you and would usually provide helpful message indicating the location, reason and possible solutions.

Unless these errors are fixed the code you will not be able to run the code. Fortunately, your IDE is able to point out most of these errors during coding.

```
>>> name = 'Narendra"  
      File "<stdin>", line 1  
        name = 'Narendra"  
            ^  
SyntaxError: unterminated string literal (detected at line 1)
```

Runtime Errors:

These errors occur while the programming is running. These can be caused by various, such as unintended user input, division by zero, trying to access as an unidentified variable or running out of memory.

As in the case of Syntax Errors, for Runtime Errors, the python interpreter will also indicate the location and reason of these errors.

```
>>> if 2 < 3:  
...   print('2 is less than 3')  
      File "<stdin>", line 2
```

```
print('2 is less than 3')  
^
```

IndentationError: expected an indented block after 'if' statement on line 1

Logical Errors:

These are the trickiest errors to find, because the interpreter will not produce any error messages. The only way than you will know that there are logical errors is by a formalized testing process or from user feedback (this is not a good sign of software quality).

If the error is easily reproducible then there is a better chance of debugging than if it is not.

Using print statement

The technique of locating errors, involves you understanding the coding intimately. You have to inserting a print statement as close as possible to the erroneous line.

```
>>> def add(x, y):  
...     return x * y                #this line is incorrect  
  
>>> print(add(3, 4))  
13
```

```
>>> def add(x, y):  
...     result = x + y              #this line is now correct  
...     print(f'{x} + {y} = {result}') #this prints the expected value  
...     return result  
  
>>> print(add(3, 4))  
7
```

Using logging

Using Python Debugger (pdb)

Using IDE Debugger (VS Code)

Setting breakpoints

Stepping through code

Examining Variables

ITERATORS

An Iterator is an object that lets you loop through a collection, like a list or tuple, one item at a time. An iterator maintains state about where it is in the sequence and knows how to get the next item.

a – The Iterator Protocol

To be considered an iterator, an object must implement two special methods collectively known as the iterator protocol:

`__iter__()` -> returns the iterator object itself.

`__next__()` -> returns the next item. If there is nothing left, it raises **StopIteration**.

b – Using Built-in Iterator

Many of Python's built-in data types (lists, tuples, strings, dictionaries, sets) are iterables—objects capable of returning their own iterator via the `iter()` function. For example

```
>>> ports = [80, 23, 20]
>>> it = iter(ports)

>>> print(next(it))
>>> print(next(it))
```

```
>>> print(next(it))
# next(it) now would raise StopIteration
```

c – Practical Benefits

- **Memory Efficiency:** Iterator fetch items one by one. This is especially useful for very large datasets or infinite sequences because you don't need to load the entire collection into memory at once.
- **Lazy Evaluation:** They compute the next item only when it is requested.
- **Clean Code:** They provide a clean and Pythonic way to iterate over sequence.
- **Pause and Resume Iteration:** You may pause and resume iteration, which is difficult with standard loops such as serving items one at a time over a network or processing data in batches.
- **Generators:** Python generators are a special kind of iterators that are even easier to create using functions and the yield keyword. They automatically handle the iterator protocol.

They're also the foundation for Python's for loops and many built-in functions like `map()`, `filter()`, and `zip()`.

The iterator pattern is everywhere in Python - when you use a for loop, Python automatically creates an iterator from the iterable and calls `next()` on it until `StopIteration` is raised.

d – Iterables vs Iterators

An **iterable** is any object that can return an **iterator**. Most python collections like lists, tuples, sets, strings, dictionaries etc. are **iterable**, meaning you can loop over them. But they do not become **iterators** until you call `iter()` on them.

```
# List is iterable, not an iterator
my_list = [1, 2, 3]
print(hasattr(my_list, '__iter__')) # True
print(hasattr(my_list, '__next__')) # False

# Getting an iterator from an iterable
my_iter = iter(my_list)
```

```
print(hasattr(my_iter, '__iter__')) # True
print(hasattr(my_iter, '__next__')) # True
```

e – Creating Custom Iterators

You can create your own iterators by implementing the iterator protocol:

```
class CountUp:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current < self.end:
            result = self.current
            self.current += 1
            return result
        else:
            raise StopIteration

# Usage
counter = CountUp(1, 4)
for num in counter:
    print(num) # Prints 1, 2, 3
```

f – When not to Prefer Iterators

- For simple, small, or static collections where you need to traverse the data multiple times, standard for loops over lists or other built-in iterables are simpler and more readable.
- If you need to reset and reuse the collection, iterables (like lists) are better, since iterators are exhausted after one pass.

GENERATORS

A generator is a simpler way to create iterator without writing a class with `__iter__()` and `__next__()`. Instead, you just write a function with `yield` instead of `return`.

When python sees a `yield`, it paused the function, saves its state, and resumes where it left off the next time you call `next()`.

```
def count_up(start, end):
    current = start
    while current < end:
        yield current
        current += 1

# Using the generator function
for num in count_up(1, 4):
    print(num) # Prints 1, 2, 3
```

Generator Expressions

Similar to list comprehension, you can create generator using generator expression with parenthesis

```
squares = (x**2 for x in range(5))
# Using the generator expression
for square in squares:
    print(square) # Prints 0 1 4 9 16
```

USING PIP

Default Installation

Install a specific version

Install Extra

Install from Source

Install from Version Control

Install from other Package Indexes

Upgrading packages

Using a requirements file

Freezing dependencies

<https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/>.

COPILOT

GitHub Copilot is an AI-powered coding assistant developed by GitHub and OpenAI. It helps you write code faster by suggesting entire lines or blocks of code as you type, based on the context of your current file and project. It supports many programming languages and is especially useful for repetitive tasks, learning new APIs, and exploring unfamiliar codebases.

.

a – Installing Copilot

You can use the venv module that is part of python 3.3+. There are also two third-party tools that

FYI

Using AI powered tools to increase programmer productivity is becoming the norm in most software houses.

b – Enabling Copilot

c – Disabling Copilot

does the same job: `virtualenv` and the more powerful `pipenv` both of which we will not be covering in this course.

d – Tips of using Copilot

Copilot can be a powerful tool when used effectively. Below are some tips to help you get the most out of it.

i – Write clear comments

Use natural language comments to describe what you want. Copilot uses these as cues to generate relevant code.

```
// Create a function that returns the Fibonacci sequence up to n
```

ii – Use descriptiveWrite clear comments

Use natural language comments to describe what you want. Copilot uses these as cues to generate relevant code.

```
// Create a function that returns the Fibonacci sequence up to n
```

- Use comments to guide Copilot (e.g., `// create a function to sort an array`).
- Press `Tab` to accept suggestions, or `Esc` to dismiss.
- You can cycle through suggestions using `Alt + [` or `Alt + ]`.

VIRTUAL ENVIRONMENTS

Python applications will often use packages and module that are not be a part of the default installation. If you are using two applications that require different versions of a particular library then you need to have both versions of the library. Often these different libraries are not able to coexist with each other.

The solution is to create separate virtual environments which is actually a self-contained directory for the application space. Each environment has its own python binary and can have its own independent set of installed packages. Libraries in these environments are isolated from other virtual environments and from the global python installation.

Using virtual environments limits the impact of package installation on the overall python distribution and prevents version clashes between projects. It makes it easier to reproduce an existing working environment and for maintaining clean and manageable development setups.

a – Creating Virtual Environments

You can use the venv module that is part of python 3.3+. There are also two third-party tools that does the same job: `virtualenv` and the more powerful `pipenv` both of which we will not be covering in this course.

FYI

To create a virtual environment in an existing folder, go one folder down and use the folder's name in creation.

The following statements walks you through the process of creating a virtual environment called `myenv` in the `c:\progs\new100\labs` folder on a windows machine. For a linux or Mac machine you will have to figure out the corresponding statements.

```
C:\progs\new100\labs>dir myenv
Volume in drive C has no label.
Volume Serial Number is 84F9-DF8C

Directory of C:\progs\new100\labs

File Not Found

C:\progs\new100\labs>python -m venv myenv
```

```
C:\progs\new100\labs>dir myenv
Volume in drive C has no label.
Volume Serial Number is 84F9-DF8C

Directory of C:\progs\new100\labs\myenv

2025-02-22  09:51 PM    <DIR>          .
2025-02-22  09:51 PM    <DIR>          ..
2025-02-22  09:51 PM    <DIR>          Include
2025-02-22  09:51 PM    <DIR>          Lib
2025-02-22  09:51 PM                88 pyvenv.cfg
2025-02-22  09:52 PM    <DIR>          Scripts
                1 File(s)                88 bytes
                5 Dir(s)  1,672,396,365,824 bytes free
```

After running `python -m venv myenv` at the command prompt, the folder `myenv` is created if it does not already exist and the necessary files to contain the virtual environment. Within this folder are three sub-folders and a file `pyvenv.cfg`.

It is possible to use a python version that is different from the default one in your workspace. First you will have to install the appropriate version on your machine and take a note of the path of the python binary. The command to create the virtual environment must be invoked with this binary e.g. `c:\python\python3.12\python -m venv myenv`.

Interesting fact

It is possible to have multiple versions of python installed and functioning properly on a single machine.

b – Activating the Virtual Environment

The script to activate the environment, does amongst other things changes the command prompt and give you a clean python environment (i.e. only the default python installation).

b.1 – On Windows

```
C:\progs\new100\labs>myenv\Scripts\activate

(myenv) C:\progs\new100\labs>
```

b.2 – On macOS/Linux

```
source myenv\Scripts\activate
```

c – Installing Packages

The preferred way of installing packages is by using pip (preferred installer program). We run `pip list` to display all the installed packages. Only two packages show up.

Reminder

You must activate the virtual environment before running pip.

We run `pip install numpy`. This installs the latest version of the numpy package. Running `pip list` verifies that the numpy package was installed.

```
(myenv) C:\progs\new100\labs>pip list
Package      Version
-----
pip          22.2.2
setuptools   63.2.0

(myenv) C:\progs\new100\labs>pip install numpy
Collecting numpy
  Downloading numpy-2.2.3-cp310-cp310-win_amd64.whl (12.9 MB)
    ----- 12.9/12.9 MB 40.9 MB/s eta
0:00:00
Installing collected packages: numpy
Successfully installed numpy-2.2.3

(myenv) C:\progs\new100\labs>pip list
Package      Version
-----
numpy        2.2.3
pip          22.2.2
setuptools   63.2.0
```

c.1 – Batch Installation

If you have to install multiple packages or if you want to re-create an existing environment you can create a text file containing the packages and versions.

c.1.1 – Creating the requirements file

Run the pip command to create a list of the current packages in an environment.

```
C:\progs\new100\labs>pip freeze > requirements.txt
```

The resulting requirements.txt is shown below:

```
blinker==1.9.0
click==8.3.1
colorama==0.4.6
Flask==3.1.2
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.3
Werkzeug==3.1.4
```

After examining the above file, you will realize that you can also create this file manually.

c.1.2 – Creating the requirements file

Run the pip command to create a list of the current packages in an environment.

```
C:\progs\new100\labs>pip install -r requirements.txt
```

d – Deactivating the Virtual Environment

Deactivating the virtual environment reverts the prompt and restore the original machine working environment.

```
(myenv) C:\progs\new100\labs>myenv\Scripts\deactivate
C:\progs\new100\labs>
```

e – Deleting a Virtual Environment

To delete the virtual environment, simply delete the containing folder. You can do this from the command prompt or from Windows Explorer.

This will permanently remove the contents of the folder!

FYI

If you need to keep your python scripts, then delete the following folders: Include, Lib, Scripts, share and the file pyvenv.cfg.

```
C:\progs\new100\labs>rmdir /s /q myenv
```

```
C:\progs\new100\labs>dir myenv
```

```
Volume in drive C has no label.
```

```
Volume Serial Number is 84F9-DF8C
```

```
Directory of C:\progs\new100\labs
```

```
File Not Found
```

For macOS/Linux, use the following command to delete the virtual environment.

```
rm -rf myenv
```

REGULAR EXPRESSION

`match()`

Determines if the pattern matches at the beginning of the string. Returns None if no match could be found. If it is successful, a match object instance is returned.

```
>>> text = 'Hot cross buns, hot cross buns.'
>>> pattern = re.compile('Hot')    #this step is mostly ignored
>>> if re.match(pattern, text):
...     print('match found')
... else:
...     print('match not found')
...
match found

>>> if re.match(r'hot', text):      #hot is not at the start of the string
...     print('match found')
... else:
...     print('match not found')
...
match not found
```

`search()`

Scan through a string looking for any location where the pattern matches. Returns None if no match could be found. If it is successful, a match object instance is returned of the first match.

```
>>> text = 'Hot cross buns, hot cross buns.'
>>> result = re.search('buns', text)
>>> result.group(0)
'buns'
```

```
>>> result.start()
10
>>> text[10:]
'buns, hot cross buns'
```

findall()

Finds all substring where the pattern matches and returns them as a list

```
>>> text = 'Hot cross buns, hot cross buns.'
>>> result = re.search('[Hh]ot', text)
>>> result
['Hot', 'hot']
```

finditer()

Finds all substring where the pattern matches and returns them as an iterator.

```
>>> text = 'Hot cross buns, hot cross bins. 1 a penny 2 a penny, hot cross buns'
>>> matches = re.finditer('ot', text)
>>> for m in matches:
...     print(m)
...
<re.Match object; span=(1, 3), match='ot'>
<re.Match object; span=(17, 19), match='ot'>
<re.Match object; span=(54, 56), match='ot'>
```

sub()

Replaces a pattern with a string.

```
text = 'This sentence has extra spaces.'
pattern = r'\s+'

clean_text = re.sub(pattern, " ", text).strip()
print(clean_text)
'This sentence has extra spaces'
```

EXCEPTION HANDLING

.

TESTING

<https://docs.python.org/3/library/unittest.html>

Types of Testing

Unit Testing

Integration Testing

Functional Testing

Regression Testing

End-to-End Testing

Tools

unittest

pytest

doctest

Basic Testing Workflow

Write Test:

Run Test:

Analyze Results:

Fix Bugs:

Refactor:

Repeat:

MODULES AND PACKAGES

a – Module

A module is normally a single text file (with extension `.py`) containing python statements. It can define classes, functions and variables and it can contain runnable code. As your program grows in size and complexity, you might find it convenient to partition your code into separate files. You are able to reference any file as a module. This is an extremely simple way to informally share code or to borrow code from other projects.

Each module has its own private namespace, therefore the author does not have to worry about accidental name clashes with other module. From a module you may import other modules.

Consider the following python file. There are three objects: a float called `pi`, a function called `greet` and another function called `circumference`.

```
'''
module01.py

This module serves to illustrate the simple
use of modules in the python language.
'''

pi = 3.141_592

def greet(name: str) -> str:
    return f'Hello {name}'

def circumference(radius: float) -> float:
    return 2 * pi * radius
```

a.1 – Importing the module

To access the statements in the about file, you have to use an import statement. The import statement read the file (module01.py must be in the current folder) and loads all the objects excepting those beginning with an underscore (_) into the current environment for immediate usage.

The `dir()` function can be used to find out all the names that are defined in a module.

```
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__']
>>> import module01
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'module01']
>>> module01.greet('Narendra')
'Hello Narendra'
```

a.2 – Import using alias

If the module has been updated and now has a new name, then your existing code that refers to the original module will not work. A fix is to use an alias when importing.

```
>>> import module01 as utils
>>> utils.circumference(10)
62.83184
```

The previous techniques of importing modules are those that are recommended by seasoned developers, because the imported objects are contained within the namespace `module01` and `utils` respectively.

a.3 – Import only selected objects

The above import is good because it does not pollute your environment. However, it is tedious to work with. Another option is to import only the objects that you are going to use.

```
>>> from module01 import greet
>>> greet('Ilia')
'Hello Ilia'
```

a.4 – Import everything

You can go one step further than the above.

```
>>> from module01 import *
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'circumference', 'greet', 'pi']
>>> circumference(10)
62.83184
>>> greet('Ilia')
```

```
'Hello Ilia'
```

```
>>> print(pi)
```

```
3.141592
```

This type of import is not recommended for large packages because it pollute the environment and may cause name clashes.

b – Package

A package is a collection of modules. Amongst the modules, there must be an `__init__.py` file so that the python interpreter will process the package properly. If the modules are contained in sub-folders, then each sub-folder must have an `__init__.py` file.

c – Library

A library is a collection of packages.

d – Framework

Typically, a framework is a collection of libraries.

EXTENDING PYTHON

Although python sports a comprehensive list of features, sometimes you might want to use features of another language because of speed, access to low-level system features, specialized language features (such as concurrency or static typing), or simply that the required functionality is already implemented.

In this exercise, we will see how to access C# code from within python.

Install the Python.Net package

This package allows you to interact with .Net libraries written in C#.

Fun fact

Most popular languages can be accessed from python.

```
pip install pythonnet
```

Write the C# code

We will implement a simple Greet method

```
//lec33_library.cs
using System;
namespace MyNamespace
{
    public class MyClass
    {
        public string Greet(string name)
        {
            // return $"Hello {name}"; //this does not work because of my
            compiler version
            return "Hello " + name;
        }
    }
}
```

Compile to a DLL

Use the C# compiler to build the library file lec33_library.dll.

```
csc /target:library lec33_library.cs
```

Load the library and invoke the Greet() method

```
>>> import clr
>>> clr.AddReference('lec33_library')
<System.Reflection.RuntimeAssembly object at 0x000001FE2ADE8200>
>>> from MyNamespace import MyClass
>>> obj = MyClass()
>>> obj.Greet('Narendra')
'Hello Narendra'
```

Other languages

For most of the popular compiled languages (rust, c and c++) the workflow is the same:

1. Write the code.
2. Compile (to a library).
3. Use the appropriate python package to host the library.
4. Use the required feature.

Some popular that were not written entirely in python are but depend on other languages for performance, functionality or compatibility with existing systems are: TensorFlow, PyTorch, Matplotlib, OpenCV2, Pandas, Numpy and SciPy. This hybrid approach allows python to remain simple and easy to use while leveraging the power of other languages for specialized tasks.

Fun fact

NumPy and SciPy uses Fortran while TensorFlow and PyTorch uses CUDA!

When not to extend python

While there are many benefits of extending python with other language, sometimes there are trade-offs:

- **Complexity:** Managing multiple languages and build system can increase complexity of the sdlc.
- **Debugging:** Debugging across language boundaries can be challenging
- **Maintenance:** Ensuring compatibility between languages and keeping dependencies up-to-date can be time-consuming.
- **Overhead:** For small projects or tasks where performance is not critical, the overhead may not be justified.

KEYWORDS

Keywords are reserved words that have special meanings and purposes in the python language. They cannot be used as variable names, function names, or any other identifiers.

There are currently 35 keywords (as of Python 3.13). These keywords define the syntax, control flow, and behavior of Python programs.

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

Fun fact

Most popular languages can be
accessed from python.

0 – GLOSSARY

Assignment

Indexable

Any object that you can access its elements by using square brackets with an index/key. It must the `__getitem__()` method. Some common objects that are considered indexable are `list`, `tuple`, `string` and `dict`.

A set is iterable but not indexable.

A dict is indexed via a key.

Iterable

Any Python object that you can loop over with a for-loop. It must implement the `__iter__()` method with indices starting from 0. Some common objects that are considered indexable are `list`, `tuple`, `string`, `set`, `dict` and `file` object.

Sequences

An ordered collection that supports element access by integer indices. It must implement `__getitem__()` with integer indices and `__len__()`.

Most also support slicing.

Every sequence is iterable and indexable, but not every iterable is a sequence.

Collections

A group of objects stored together. It must be iterable, sized (`__len__()`) and support membership testing (`__in__()`). Examples include `list`, `tuple`, `set` and `dict`.

It is a container with a size and membership

Mappings

In Python, a mapping is a type of collection that associates keys with values. Instead of accessing items by position (like sequence do), you access them by a key. A dict is the most common mapping type.

The keys must be unique and hashable (immutable types like string, numbers and tuples of immutables).

Values can be on any type.

The order of insertion is preserved (since Python 3.7+)

Method

Function

Overloading

Inheritance

Type Annotation

Docstring

Comments

Repl

IDE

Debugging

Operator

Keyword

Module

Import

Algorithm

A step-by-step set of instructions to solve a problem.

Boolean (bool)

A data type with only two values: True or False.

Collection

A container that groups multiple values (e.g., list, tuple, set, dict).

Comment

A note in code that Python ignores. Starts with #. Example:

```
# This is a comment
```

Data Type

The kind of value stored in a variable, such as int, float, str, or bool.

Debugging

The process of finding and fixing errors (bugs) in your code.

Dictionary (dict)

A mapping type that stores key–value pairs. Example:

```
student = {"name": "Alice", "age": 20}
```

Docstring

A special string inside triple quotes (""") that documents what a function, class, or module does.

Function

A reusable block of code that performs a task. Defined with def.

```
def greet(name):  
    return "Hello " + name
```

Method

A function that belongs to an object and is called using dot notation. Example:

```
text = "hello"  
  
print(text.upper()) # upper() is a method
```

Indentation

Spaces at the start of a line that define code blocks in Python. Required!

Inheritance

A way for one class to get properties and methods from another class. Helps reuse code.

Iterable

Any object you can loop over with `for ... in`.

Keyword

A reserved word in Python that has a special meaning, like `if`, `for`, `class`, `def`.

List

A mutable, ordered collection of items. Example:

```
nums = [10, 20, 30]
```

Loop

Repeats code multiple times. Examples: `for`, `while`.

Module

A file that contains Python code (functions, classes, variables) you can reuse.

Import

The way to bring in code from a module. Example:

```
import math
```

```
print(math.sqrt(16))
```

Mutable

An object that can be changed after creation (e.g., lists, dicts).

Operator

A symbol that performs an action, like +, -, *, /, %.

Overloading (operator or method)

Giving special meaning to an operator or method depending on the data type.

Example:

```
print(2 + 3)    # adds numbers
```

```
print("a" + "b") # concatenates strings
```

Sequence

An ordered collection with index positions (e.g., string, list, tuple).

String (str)

Text data in quotes. Example: "hello"

Tuple

An ordered collection like a list, but immutable (cannot change).

```
point = (3, 4)
```

Type Annotation

A way to specify the type of a variable, parameter, or return value. Example:

```
def add(x: int, y: int) -> int:  
  
    return x + y
```

Variable

A name that stores a value. Example:

```
age = 18
```

Basically, there are four ways of running python code: from a notebook (jupyter is a local version while google colab is an online version), from the python repl, invoking the python executable and passing the code file and finally setting the permission mode of the code file. Each method has its convenience and advantage in a particular situation.

Fun fact

A Pythonista is a term used to describe an enthusiast, expert, or dedicated user of the Python programming language.

Below is a sample session in the python **repl** in a windows 10 environment:

```
Copyright (C) Microsoft Corporation. All rights reserved.
```

This is probably the best and the most professional way of running or distributing your code to everyone from the novice user to the seasoned developer.

This is only possible in the Linux/Unix or MacOS operating systems that supports file permissions.

Interesting fact

Although this technique only works with *nix OS'es, you may get the same effect in the Windows world by creating a batch or powershell script with the appropriate commands.

```
my_number = 4
my_number += 2      #my_number is now is 6
```

2.j.7 – Operator precedence

If you have an expression with more than one operator, then python uses a set of internal rules to decide the order that the computation must follow. The table below list the various operators in descending order of precedence.

Operator	Description
()	Parentheses
**	Exponentiation
+x -x ~x	Unary plus, unary minus, and bitwise NOT
* / // %	Multiplication, division, floor division, and modulus
+ -	Addition and subtraction
<< >>	Bitwise left and right shifts
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
== != > >= < <= is is not in not in	Comparisons, identity, and membership operators

not	Logical NOT
and	AND
or	OR